
پردازش تصویر در دهه آینده

از عمق ریاضیات تا مهندس حرفه‌ای

کتاب - جزوه درس کارشناسی ارشد و آزمایشگاه پژوهش محور

دوازدهم فصل

تصویر مولد مدل‌های

مدل‌های VAE، VQ-VAE، GAN تا واریاسیونی استنباط و خودرنگ‌گذار از
جدید توکنی مولدهای و انرژی محور مدل‌های نرمال‌ساز، جریان‌های خودرگرسیو،

مدل بداند باید دانشجو نیست. چشم‌نواز تصویر چند تولید فقط فصل هدف
می‌کند، کمینه را کرانی یا فاصله چه هدف تابع می‌آموزد، را توزیعی چه
چرا و می‌شود داده تشخیص چگونه فروپاشی دارد، هزینه‌ای چه نمونه‌گیری
نیست. کافی موفقیت ادعای برای منتخب تصویر چند یا عددی معیار یک

نسخه تفصیلی درس - تابستان ۱۴۰۵ / ۲۰۲۶

قابل کامپایل با XeLaTeX + TeXstudio + TeX Live

یادداشت روش شناختی فصل دوازدهم

فصل‌های پیشین به استخراج ویژگی، شبکه‌های کانولوشنی و نمایش‌های خودنظارتی پرداختند. در آن فصل‌ها، مدل عمدتاً یک نگاشت پیش‌بین بود: تصویر را می‌گرفت و برچسب، ویژگی یا ماسک می‌ساخت. مدل مولد یک پرسش بنیادی‌تر می‌پرسد: چه سازوکار احتمالاتی می‌تواند خود داده را تولید کند؟ پاسخ این پرسش می‌تواند به صورت چگالی صریح، متغیر نهفته، بازی خصمانه، فاکتورگیری خودرگرسیو، تبدیل وارون‌پذیر یا میدان انرژی بیان شود. این صورت‌بندی‌ها از نظر تابع هدف، نوع استنباط، کیفیت نمونه، پوشش توزیع و هزینه محاسباتی یکسان نیستند.

در این فصل، اصطلاح «مدل مولد» برای هر شبکه‌ای که تصویری خروجی می‌دهد به کار نمی‌رود. یک مدل مولد باید درباره توزیع داده ادعایی مشخص داشته باشد؛ مثلاً چگالی یا کران درست‌نمایی ارائه کند، نمونه‌های خود را به عنوان pushforward یک توزیع پایه تعریف کند، یا از طریق یک منتقد فاصله میان توزیع داده و مدل را کاهش دهد. بنابراین هر معماری با پنج پرسش سنجیده می‌شود:

۱. متغیر تصادفی پایه و فضای نمونه چیست؟
۲. مدل چگالی صریح دارد یا فقط نمونه تولید می‌کند؟
۳. تابع هدف دقیقاً چه کمیتی را کمینه یا بیشینه می‌کند؟
۴. استنباط، بازسازی و نمونه‌گیری چگونه انجام می‌شوند؟
۵. شکست غالب آن چیست و با چه آزمونی آشکار می‌شود؟

هدف فصل

این فصل شش خروجی هم‌زمان دارد:

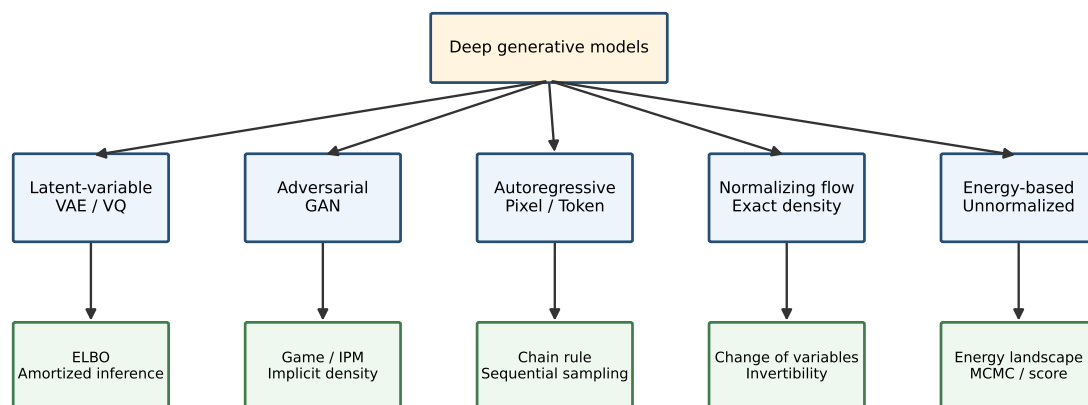
۱. مبانی احتمال: دانشجو بتواند درست‌نمایی، واگرایی، pushforward تغییر متغیر و متغیر نهفته را با نمادگذاری دقیق بنویسد.
۲. استنباط واریاسیونی: دانشجو بتواند ELBO را استخراج و شکاف واریاسیونی، بازپارامتردهی و فروپاشی پسین را تحلیل کند.
۳. بازی خصمانه: دانشجو بتواند تمایزگر بهینه، ارتباط GAN با JS، نسخه غیر اشباع، WGAN و جریمه گرادیان را توضیح دهد.

۴. مدل‌های چگالی صریح: دانشجو بتواند فاکتورگیری خودرگرسیو و فرمول تغییر متغیر در جریان‌های نرمال‌ساز را پیاده‌سازی کند.

۵. توان مهندسی: دانشجو بتواند نسخه آموزشی VAE، VQ-VAE، GAN، WGAN-GP، PixelCNN و RealNVP را اجرا و عیب‌یابی کند.

۶. ارزیابی صادقانه: دانشجو میان کیفیت، پوشش، درست‌نمایی، حفظ کردن داده، وفاداری شرطی و هزینه تمایز بگذارد.

A unified taxonomy: what is normalized, what is inferred, and how are samples obtained?



شکل ۱: رده‌بندی یکپارچه مدل‌های مولد عمیق. تفاوت اصلی فقط معماری نیست؛ بلکه نوع نرمال‌سازی، استنباط، تابع هدف و سازوکار نمونه‌گیری است.

نقشه فشرده فصل

محور	پرسش و خروجی آموزشی
احتمال مولد خودرمزگذار VAE	p_{θ} ، p_{data} ، pusfhorward چگالی صریح و ضمنی چه تفاوتی دارند؟ بازسازی چه نسبتی با PCA، منیفلد و مدل مولد دارد؟ ELBO چگونه استخراج می شود و بازپارامتردهی چگونه گرادیان را از نمونه گیری عبور می دهد؟
نسخه های واریاسیونی	β -VAE، IWAE، WAE، InfoVAE و مدل های سلسله مراتبی کدام بخش مدل را تغییر می دهند؟
کوانتیزه سازی	VQ-VAE/VQGAN چگونه تصویر را به الفبای گسسته تبدیل می کند و collapse codebook چیست؟
GAN	تمایزگر بهینه چه نسبت چگالی ای می آموزد و چرا آموزش یک بازی دینامیکی است؟
پایداری خصمانه	WGAN، قید $\text{gradient Lipschitz}$ ، $\text{nor-spectral penalty}$ ، R1 malization و TTUR چه می کنند؟
معماری های GAN	DCGAN، شرطی، pix2pix، CycleGAN، BigGAN و StyleGAN1-3 چه پیشرفت هایی دارند؟
خودرگرسیو	قاعده زنجیره چگونه چگالی دقیق می دهد و چرا نمونه گیری پیکسلی کند است؟
مولدهای توکنی	VQGAN+Transformer، MaskGIT، LlamaGen، VAR و های tokenizer فشرده چه مبادله ای دارند؟
جریان ها	تغییر متغیر، RealNVP coupling ، Glow، MAF/IAF و جریان پیوسته چگونه درست نمایی دقیق می دهند؟
انرژی	چرا ثابت نرمال سازی دشوار است و فاز مثبت/منفی، MCMC و matching score چه نقشی دارند؟
ارزیابی	ELBO، FID bits/dim ، KID، precision/recall حفظ داده و ارزیابی انسانی چگونه ترکیب می شوند؟
کتابخانه ها	MMagic، Diffusers PyTorch، مخزن StyleGAN3 و ابزارهای معیار در چه سطحی استفاده می شوند؟
پل به فصل بعد	جریان تطبیقی، score و مدل های نهفته چگونه به مدل های انتشار و انتقال پیوسته می رسند؟

فهرست مطالب

آ	یادداشت روش شناختی فصل دوازدهم
پ	نقشه فشرده فصل
۱	۱۲ مدل‌های مولد تصویر
۱	۱۰۱۲ مدل مولد دقیقاً چیست؟
۲	۱۰۱۰۱۲ سه هدف متفاوت که نباید مخلوط شوند
۲	۲۰۱۰۱۲ Pustforward یک توزیع پایه
۲	۲۰۱۲ مبانی احتمال و هندسه فاصله میان توزیع‌ها
۳	۱۰۲۰۱۲ پیشینه‌سازی درست‌نمایی و آنتروپی متقاطع
۳	۲۰۲۰۱۲ واگرایی KL و نامتقارن بودن آن
۳	۳۰۲۰۱۲ واگرایی Jensen-Shannon
۳	۴۰۲۰۱۲ فاصله Wasserstein و IPM
۴	۳۰۱۲ خودرمزگذار: خط مبنا، نه الزاماً مدل مولد
۵	۱۰۳۰۱۲ چه زمانی خودرمزگذار به مولد نزدیک می‌شود؟
۵	۴۰۱۲ مدل متغیر نهفته و VAE
۵	۱۰۴۰۱۲ استخراج ELBO از هویت دقیق
۶	۲۰۴۰۱۲ بازپارامتردهی گاوسی
۷	۳۰۴۰۱۲ KL بسته برای گاوسی قطری
۷	۴۰۴۰۱۲ مدل مشاهده و معنای زیان بازسازی
۷	۵۰۴۰۱۲ تجزیه نرخ - اعوجاج
۱۰	۵۰۱۲ فروپاشی پسین و عیب‌یابی VAE
۱۰	۱۰۵۰۱۲ چرا رخ می‌دهد؟
۱۰	۲۰۵۰۱۲ آزمون‌های تشخیصی

۱۱	 راهکارها ۳۰۵.۱۲
۱۱	 VAE خانواده گسترده ۶۰۱.۱۲
۱۱	 IWAE و کران چند نمونه‌ای ۱۰۶.۱۲
۱۱	 posterior تجمعی و تطبیق ۲۰۶.۱۲
۱۲	 مدل‌های سلسله‌مراتبی ۳۰۶.۱۲
۱۲	 VAE شرطی ۴۰۶.۱۲
۱۳	 VQ-VAE: متغیر نهفته گسسته و کد تصویری ۷۰۱.۱۲
۱۳	 collapse codebook و Perplexity ۱۰۷.۱۲
۱۴	 codebook EMA به روزرسانی ۲۰۷.۱۲
۱۴	 VQGAN و حد بالای کیفیت tokenizer ۳۰۷.۱۲
۱۶	 GAN: یادگیری توزیع به وسیله بازی تمایز ۸۰۱.۱۲
۱۷	 استخراج تمایزگر بهینه ۱۰۸.۱۲
۱۷	 زیان غیر اشباع مولد ۲۰۸.۱۲
۱۸	 چرا GAN ناپایدار است؟ ۹۰۱.۱۲
۱۸	 فروپاشی مد ۱۰۹.۱۲
۱۹	 برتری بیش از حد تمایزگر ۲۰۹.۱۲
۱۹	 WGAN و هندسه Lipschitz ۱۰۰.۱۲
۱۹	 چرا clipping weight نامناسب است؟ ۱۰۱.۱۲
۱۹	 penalty Gradient ۲۰۱.۱۲
۲۰	 normalization Spectral ۳۰۱.۱۲
۲۰	 TTUR و regularization lazy، ۱Rf ۴۰۱.۱۲
۲۲	 معماری‌های مهم GAN ۱۰۱.۱۲
۲۲	 DCGAN: قرارداد معماری پایه ۱۰۱.۱۲
۲۲	 discriminator projection و شرطی سازی ۲۰۱.۱۲
۲۳	 PatchGAN و pix۲pix ۳۰۱.۱۲
۲۳	 CycleGAN و عدم شناساپذیری ۴۰۱.۱۲
۲۳	 BigGAN و مقیاس ۵۰۱.۱۲
۲۳	 StyleGAN و هندسه فضای نهفته ۲۰۱.۱۲
۲۴	 Modulation و demodulation در StyleGAN ۱۰۲.۱۲
۲۴	 alias و حذف ۳StyleGAN ۲۰۱.۱۲
۲۴	 Inversion و ویرایش ۳۰۱.۱۲
۲۵	 مدل‌های خودرگرسیو: چگالی دقیق با قاعده زنجیره ۱۳۰.۱۲

۲۵	convolution masked و PixelCNN PixelRNN، ۱۰.۱۳.۱۲
۲۶	likelihood پیکسل رنگی، ۲۰.۱۳.۱۲
۲۷	۱۴.۱۲ از پیکسل به توکن: مولدهای تصویری خودرگرسیو جدید
۲۸	Transformer + VQGAN، ۱۰.۱۴.۱۲
۲۸	MaskGIT و decoding موازی تکراری، ۲۰.۱۴.۱۲
۲۸	LlamaGen و prediction next-token، ۳۰.۱۴.۱۲
۲۸	VAR و prediction next-scale، ۴۰.۱۴.۱۲
۲۸	۵۰.۱۴.۱۲ توکن‌های یک‌بعدی فشرده
۲۹	۱۵.۱۲ جریان‌های نرمال‌ساز: چگالی دقیق و نگاشت وارون‌پذیر
۲۹	RealNVP در coupling Affine، ۱۰.۱۵.۱۲
۳۰	Glow، ۳۰.۱۵.۱۲
۳۰	MAF و IAF، ۳۰.۱۵.۱۲
۳۰	Dequantization، ۴۰.۱۵.۱۲
۳۱	Matching Flow و پیوسته، ۱۶.۱۲ جریان‌های
۳۲	۱۷.۱۲ مدل‌های انرژی‌محور
۳۳	dynamics Langevin، ۱۰.۱۷.۱۲
۳۳	matching Score، ۳۰.۱۷.۱۲
۳۳	۱۸.۱۲ مدل‌های هیبرید و چرا مرز خانواده‌ها محو شده است
۳۳	VAE-GAN، ۱۰.۱۸.۱۲
۳۴	WAE و Autoencoder Adversarial، ۲۰.۱۸.۱۲
۳۴	diffusion latent و Transformer + VQGAN، ۳۰.۱۸.۱۲
۳۴	VAE در posterior Flow، ۴۰.۱۸.۱۲
۳۴	۹۰.۱۲ شرطی‌سازی، کنترل و ترجمه تصویر
۳۴	normalization Conditional، ۱۰.۱۹.۱۲
۳۵	۲۰.۱۹.۱۲ تنوع شرطی
۳۵	۲۰.۱۲ فضای نهفته، درونیابی و جداسازی عوامل
۳۵	interpolation spherical و Linear، ۱۰.۲۰.۱۲
۳۵	Disentanglement بدون نظارت، ۲۰.۲۰.۱۲
۳۵	۲۱.۱۲ ارزیابی مدل مولد: یک عدد کافی نیست
۳۵	dimension per bits و ELBO Likelihood، ۱۰.۲۱.۱۲
۳۶	FID، ۲۰.۲۱.۱۲
۳۶	MMD و KID، ۳۰.۲۱.۱۲

۳۶		۴۰.۲۱.۱۲ Recall و Precision توزیعی
۳۶		۵۰.۲۱.۱۲ حفظ کردن داده و audit nearest-neighbor
۳۷		۶۰.۲۱.۱۲ عارضیابی شرطی
۳۸		۲۰.۱۲ پروتکل آزمایش بازتولیدپذیر
۳۹		۲۳.۱۲ گنجخانه‌ها و ابزارهای حرفه‌ای
۳۹		۱۰.۲۳.۱۲ PyTorch و torch.distributions
۳۹		۲۰.۲۳.۱۲ tutorial DCGAN torchvision
۳۹		۳۰.۲۳.۱۲ MMagic
۴۰		۴۰.۲۳.۱۲ مخزن رسمی StyleGAN ۳
۴۰		۵۰.۲۳.۱۲ VQModel و AutoencoderKL Ddiffusers
۴۰		۶۰.۲۳.۱۲ ابزار معیار
۴۰		۲۴.۱۲ انتخاب خانواده مناسب
۴۱		۲۵.۱۲ ۲۰۲۴ تا ۲۰۲۶
۴۲		۲۶.۱۲ الگوریتم‌های مرجع فصل
۴۳		۲۷.۱۲ آزمایش‌های پیشنهادی فصل
۴۴		۸۰.۱۲ پروژه‌های پایان فصل
۴۴		۲۹.۱۲ تمرین‌های نظری
۴۵		۳۰.۱۲ تمرین‌های کدنویسی
۴۶		۱۰.۱۲ پرامپت جامع برای Codex
۴۷		منابع منتخب و مسیر مطالعه

۵۲		آ راهنمای ساخت، اجرا و بازتولید
۵۲		۱.آ ساخت PDF
۵۲		۲.آ محیط پایتون
۵۲		۳.آ تولید شکل‌ها
۵۲		۴.آ test Smoke کدها
۵۳		۵.آ پیش‌پرواز علمی

فصل ۱۲

مدل‌های مولد تصویر

اهداف یادگیری

در پایان فصل، دانشجو باید بتواند:

- مدل مولد صریح و ضمنی را تعریف و برای هر خانواده مسیر نمونه‌گیری را رسم کند؛
- واگرایی‌های KL و JS، فاصله Wasserstein و IPM را از منظر بهینه‌سازی مقایسه کند؛
- ELBO، KL، گوسی و برآوردگر بازپارامتردهی را استخراج کند؛
- فروپاشی پسین و فروپاشی codebook را با معیارهای عددی تشخیص دهد؛
- تمایزگر بهینه GAN و مقدار تابع هدف در آن را اثبات کند؛
- WGAN-GP و normalization spectral را برای قید Lipschitz توضیح دهد؛
- یک convolution masked علی و یک coupling وارون‌پذیر بنویسد؛
- معیارهای FID/KID را از آمار ویژگی‌ها محاسبه و محدودیت آن‌ها را بیان کند؛
- یک پروتکل آزمایش منصفانه با seed، split، تعداد نمونه ثابت، بازه اطمینان و -nearest-audit neighbor طراحی کند.

۱.۱۲ مدل مولد دقیقاً چیست؟

فرض کنید تصاویر مشاهده‌شده نمونه‌هایی مستقل از توزیع ناشناخته $p_{\text{data}}(x)$ باشند. هدف مدل مولد، ساخت خانواده‌ای پارامتری p_{θ} یا یک سازوکار نمونه‌گیری G_{θ} است که رفتار توزیعی داده را تقریب بزند. این هدف با طبقه‌بندی تفاوت دارد: در طبقه‌بندی فقط مرز تصمیم مهم است، اما در تولید باید احتمال یا جرم تمام ناحیه‌های معتبر فضا توصیف شود. اگر مدل فقط چند نمونه بسیار زیبا تولید کند ولی بخش بزرگی از حالت‌های واقعی را نادیده بگیرد، توزیع را نیاموخته است.

۱۰.۱۲ سه هدف متفاوت که نباید مخلوط شوند

۱. برآورد چگالی: برای ورودی x مقدار $p_\theta(x)$ یا کرانی بر آن محاسبه شود.
 ۲. نمونه‌گیری: بتوان $x \sim p_\theta$ تولید کرد؛ حتی اگر چگالی صریح ناشناخته باشد.
 ۳. استنباط نهفته: برای تصویر مشاهده‌شده، نمایش z یا توزیع پسین $p_\theta(z | x)$ به دست آید.
- خانواده‌ها در این سه توانایی متفاوت‌اند. جریان نرمال‌ساز، چگالی و استنباط دقیق دارد؛ GAN نمونه‌گیری سریع اما چگالی ضمنی دارد؛ مدل خودرگرسیو درست‌نمایی دقیق اما نمونه‌گیری ترتیبی دارد؛ VAE استنباط سریع و یک کران واریاسیونی دارد.

۲۰.۱۲ Pufshorward یک توزیع پایه

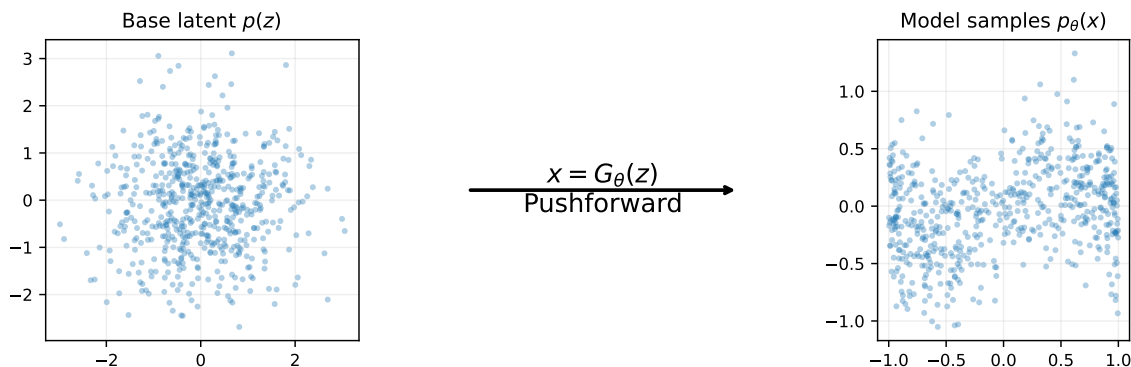
اگر $z \sim p_z$ و $x = G_\theta(z)$ باشد، توزیع مدل pufshorward توزیع پایه است:

$$p_\theta = (G_\theta)_\# p_z. \quad (۱۰۱۲)$$

این نماد یعنی برای هر مجموعه اندازه‌پذیر A ،

$$p_\theta(A) = p_z(G_\theta^{-1}(A)). \quad (۲۰۱۲)$$

اگر بعد نهفته کمتر از بعد تصویر باشد، توزیع روی یک منیفلد کم‌بعد قرار می‌گیرد و لزوماً چگالی لبگ در کل فضای پیکسل ندارد. این نکته یکی از علت‌های دشواری مقایسه چگالی‌های صریح و مدل‌های ضمنی است.



شکل ۱۰۱۲: تولید به صورت pufshorward توزیع پایه ساده با نگاشت شبکه‌ای به توزیع پیچیده داده منتقل می‌شود.

۲۰.۱۲ مبانی احتمال و هندسه فاصله میان توزیع‌ها

۱۰.۲.۱۲ پیشینه‌سازی درست‌نمایی و آنتروپی متقاطع

برای داده $D = \{x_i\}_{i=1}^n$ ، برآورد پیشینه درست‌نمایی برابر است با

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} \sum_{i=1}^n \log p_{\theta}(x_i). \quad (۳.۱۲)$$

در حد جمعیت، کمینه‌کردن منفی درست‌نمایی همان کمینه‌کردن

$$\mathbb{E}_{p_{\text{data}}}[-\log p_{\theta}(x)] = H(p_{\text{data}}) + \text{KL}(p_{\text{data}} \| p_{\theta}) \quad (۴.۱۲)$$

است. چون آنتروپی داده نسبت به θ ثابت است، ML جهت KL رو به جلو را کمینه می‌کند. این جهت به نواحی دارای جرم داده حساس است و معمولاً mode-covering توصیف می‌شود، اما تعبیر هندسی آن وابسته به خانواده مدل و ظرفیت آن است.

۲۰.۲.۱۲ واگرایی KL و نامتقارن بودن آن

$$\text{KL}(p \| q) = \int p(x) \log \frac{p(x)}{q(x)} dx. \quad (۵.۱۲)$$

اگر جایی $p(x) > 0$ و $q(x) = 0$ باشد، مقدار بی‌نهایت می‌شود. جهت معکوس $\text{KL}(q \| p)$ رفتار دیگری دارد و ممکن است به انتخاب یک مد پرچگالی تشویق کند. بنابراین عبارت «کمینه‌کردن KL بدون ذکر جهت، ناقص است».

۳۰.۲.۱۲ واگرایی Jensen-Shannon

$$m = (p + q)/2$$

$$\text{JS}(p \| q) = \frac{1}{2} \text{KL}(p \| m) + \frac{1}{2} \text{KL}(q \| m). \quad (۶.۱۲)$$

این کمیت متقارن و کراندار است؛ ولی هنگامی که تکیه‌گاه دو توزیع روی منیفدهای جدا باشد، می‌تواند اشباع شود و گرادیان مفیدی برای حرکت منیفد مولد فراهم نکند.

۴۰.۲.۱۲ فاصله Wasserstein و IPM

فاصله انتقال بهینه مرتبه یک

$$W_1(p, q) = \inf_{\gamma \in \Pi(p, q)} \mathbb{E}_{(x, y) \sim \gamma} \|x - y\| \quad (۷.۱۲)$$

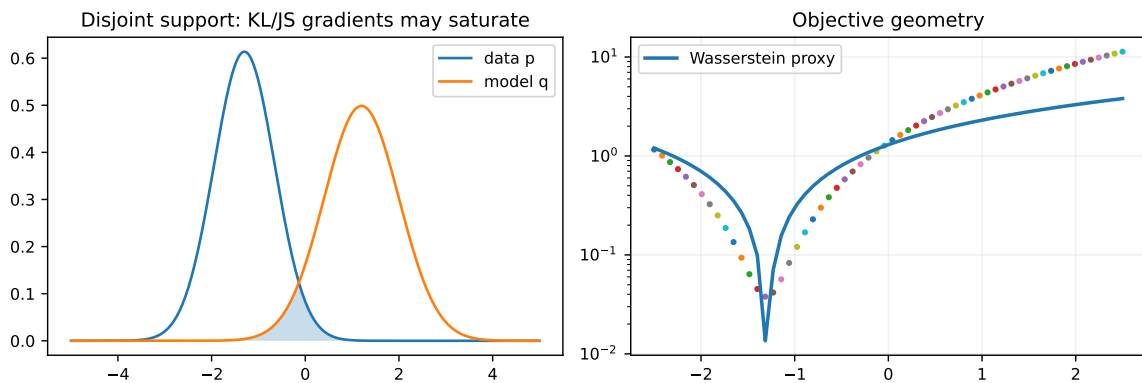
هزینه جابه‌جایی جرم را اندازه می‌گیرد. دوگان کانتورویچ-روینشتاین می‌گوید

$$W_1(p, q) = \sup_{\|f\|_L \leq 1} [\mathbb{E}_p f(x) - \mathbb{E}_q f(x)]. \quad (۸.۱۲)$$

این فرم پایه WGAN است. خانواده کلی‌تر معیارهای انتگرالی احتمال نیز به صورت

$$d_{\mathcal{F}}(p, q) = \sup_{f \in \mathcal{F}} |\mathbb{E}_p f - \mathbb{E}_q f| \quad (۹.۱۲)$$

تعریف می‌شود. انتخاب کلاس تابع $\mathcal{F}$ تعیین می‌کند مدل چه تفاوت‌هایی را قابل مشاهده بداند.



شکل ۲۰۱۲: تابع فاصله فقط یک عدد گزارش نیست؛ هندسه آن تعیین می‌کند وقتی تکیه‌گاه‌ها جدا هستند چه گرادیانی به مدل برسد.

خطای رایج

کوچک بودن یک واگرایی در فضای ویژگی یا تحت یک منتقد محدود، به معنی برابری کامل دو توزیع نیست. هر معیار فقط تفاوت‌هایی را می‌بیند که کلاس تابع یا استخراج‌گر ویژگی آن قادر به نمایش باشد.

۳.۱۲ خودرمزگذار: خط مبنا، نه الزاماً مدل مولد

خودرمزگذار تعیین‌گر شامل رمزگذار f_ϕ و رمزگشا g_θ است:

$$z = f_\phi(x), \quad \hat{x} = g_\theta(z), \quad (۱۰.۱۲)$$

و معمولاً

$$\min_{\phi, \theta} \frac{1}{n} \sum_i \ell(x_i, g_\theta(f_\phi(x_i))) \quad (۱۱.۱۲)$$

را حل می‌کند. اگر فضای نهفته کم‌بعد و نگاشت‌ها خطی باشند و زیان مربعی استفاده شود، زیر فضای بهینه با زیر فضای مؤلفه‌های اصلی ارتباط دارد. اما خودرمزگذار غیرخطی می‌تواند داده آموزش را در نواحی پراکنده فضای نهفته قرار دهد؛ در این صورت نمونه‌گیری $z \sim \mathcal{N}(0, I)$ الزاماً تصویر معتبری تولید نمی‌کند.

۱۰۳۰۱۲ چه زمانی خودرمزگذار به مولد نزدیک می‌شود؟

برای نمونه‌گیری معنی‌دار باید هندسه نهفته تنظیم شود؛ مثلاً با prior صریح، نویز، قرارداد محلی، regularization، adversarial AE، contractive autoencoder، denoising. AE و WAE هرکدام یکی از این مسیرها را دنبال می‌کنند.

۴۰۱۲ مدل متغیر نهفته و VAE

مدل مولد پیوسته را چنین می‌نویسیم:

$$p_{\theta}(\mathbf{x}, \mathbf{z}) = p(\mathbf{z})p_{\theta}(\mathbf{x} | \mathbf{z}), \quad p_{\theta}(\mathbf{x}) = \int p(\mathbf{z})p_{\theta}(\mathbf{x} | \mathbf{z}) d\mathbf{z}. \quad (12.12)$$

انتگرال حاشیه‌ای و پسین

$$p_{\theta}(\mathbf{z} | \mathbf{x}) = \frac{p(\mathbf{z})p_{\theta}(\mathbf{x} | \mathbf{z})}{p_{\theta}(\mathbf{x})} \quad (13.12)$$

برای شبکه عمیق غالباً محاسبه ناپذیرند. VAE یک توزیع استنباط تقریبی $q_{\phi}(\mathbf{z} | \mathbf{x})$ را با شبکه رمزگذار می‌آموزد؛ این پدیده را استنباط amortized می‌نامیم، زیرا هزینه بهینه‌سازی جداگانه برای هر نمونه به هزینه آموزش یک شبکه مشترک تبدیل می‌شود.

۱۰۴۰۱۲ استخراج ELBO از هویت دقیق

برای هر q_{ϕ} با تکیه‌گاه مناسب،

$$\log p_{\theta}(\mathbf{x}) = \mathbb{E}_{q_{\phi}} [\log p_{\theta}(\mathbf{x}, \mathbf{z}) - \log q_{\phi}(\mathbf{z} | \mathbf{x})] + \text{KL}(q_{\phi}(\mathbf{z} | \mathbf{x}) || p_{\theta}(\mathbf{z} | \mathbf{x})) \quad (14.12)$$

$$= \text{ELBO}(\mathbf{x}; \theta, \phi) + \text{gap variational}. \quad (15.12)$$

چون KL نامنفی است،

$$\log p_{\theta}(\mathbf{x}) \geq \mathbb{E}_{q_{\phi}(\mathbf{z} | \mathbf{x})} \log p_{\theta}(\mathbf{x} | \mathbf{z}) - \text{KL}(q_{\phi}(\mathbf{z} | \mathbf{x}) || p(\mathbf{z})). \quad (16.12)$$

جمله اول داده را بازسازی می‌کند و جمله دوم نرخ اطلاعات یا هزینه دورشدن پسین از prior است.

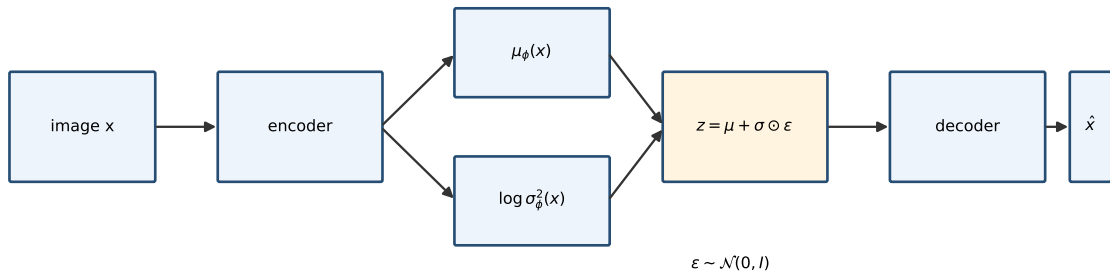
مسیر اثبات: استخراج بدون پرش ELBO

از هویت

$$\log p_{\theta}(x) = \log \frac{p_{\theta}(x, z)}{p_{\theta}(z | x)} \quad (17.12)$$

شروع کنید، امید نسبت به $q_{\phi}(z | x)$ بگیرید، سپس $\log q_{\phi}$ را جمع و تفریق کنید. جمله حاوی $\log q_{\phi} - \log p_{\theta}(z | x)$ دقیقاً KL شکاف واریاسیونی است. حذف این جمله نامنفی، کران پایین را می‌دهد.

VAE computational graph and the reparameterization path



شکل ۳.۱۲: گراف محاسباتی VAE. بازپارامتردهی، منبع تصادف را از پارامترهای شبکه جدا می‌کند تا گرادین pathwise قابل محاسبه باشد.

۲۰۴.۱۲ بازپارامتردهی گاوسی

اگر

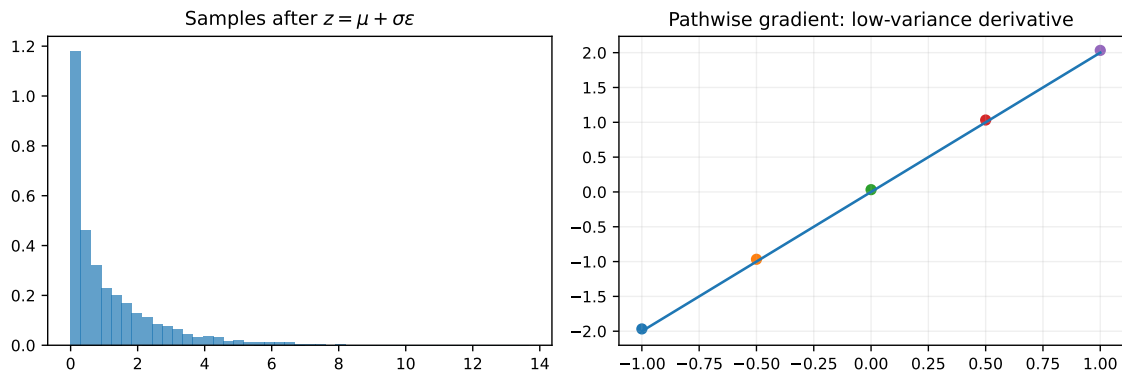
$$q_{\phi}(z | x) = \mathcal{N}(\mu_{\phi}(x), \text{diag}(\sigma_{\phi}^2(x))), \quad (18.12)$$

آنگاه نمونه را به صورت

$$\epsilon \sim \mathcal{N}(\cdot, I), \quad z = \mu_{\phi}(x) + \sigma_{\phi}(x) \odot \epsilon \quad (19.12)$$

می‌نویسیم. اکنون تصادف در ϵ مستقل از پارامتر است و مشتق از مسیر تعیین گرای بعدی عبور می‌کند. برای تابع h ,

$$\nabla_{\phi} \mathbb{E}_{q_{\phi}(z|x)} h(z) = \mathbb{E}_{\epsilon} \nabla_{\phi} h(\mu_{\phi} + \sigma_{\phi} \odot \epsilon). \quad (20.12)$$



شکل ۴.۱۲: بازپارامتردهی گاوسی نمونه‌گیری را به یک تبدیل مشتق‌پذیر از نویز استاندارد تبدیل می‌کند.

۳.۴.۱۲ KL بسته برای گاوسی قطری

برای prior استاندارد،

$$\text{KL}(q_\phi(z | x) \| \mathcal{N}(\cdot, I)) = \frac{1}{2} \sum_{j=1}^{d_z} (\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2). \quad (۲۱.۱۲)$$

در کد بهتر است شبکه $\log \sigma^2$ را خروجی دهد، آن را در بازه‌ای ایمن clamp کند و از $\sigma = \exp(\frac{1}{2} \log \sigma^2)$ استفاده شود.

۴.۴.۱۲ مدل مشاهده و معنای زیان بازسازی

زیان بازسازی انتخابی دخواه نیست؛ منفی log-likelihood مدل مشاهده است:

• Bernoulli برای داده دودویی؛ binary cross-entropy

• Gaussian با واریانس ثابت: MSE تا ضریب و ثابت؛

• Gaussian با واریانس آموختنی: جمله $\log \sigma^2$ نیز ضروری است؛

• mixture logistic discretized برای پیکسل‌های گسسته رنگی؛

• Laplace: زیان ℓ_1 ؛

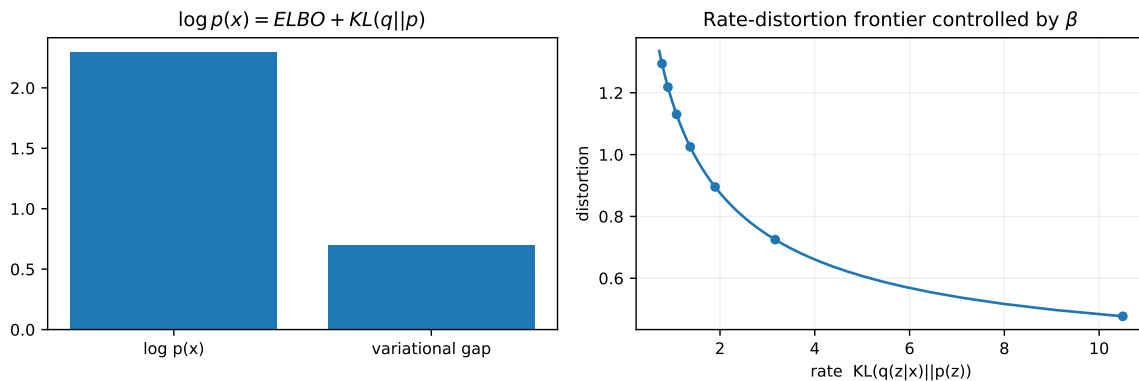
• likelihood ادراکی: مدل آماری دقیق پیکسل نیست و باید با احتیاط تفسیر شود.

۵.۴.۱۲ تجزیه نرخ - اعوجاج

نسخه β -VAE

$$\mathcal{L}_\beta = \mathbb{E}_q[-\log p_\theta(x | z)] + \beta \text{KL}(q(z | x) \| p(z)) \quad (۲۲.۱۲)$$

میان اعوجاج بازسازی و نرخ نهفته مبادله ایجاد می‌کند. افزایش β معمولاً فشرده‌سازی و جداسازی عوامل را تقویت می‌کند ولی جزئیات را کاهش می‌دهد. این تعبیر از برچسب مبهم regularization «بیشتر» دقیق‌تر است.



شکل ۵.۱۲: هویت ELBO و جبهه نرخ-اعوجاج. یک VAE خوب لزوماً کمترین خطای بازسازی را ندارد؛ باید نرخ نهفته نیز کنترل شود.

الگوریتم ۱۰.۱۲ - آموزش یک VAE گاوسی

برای هر minibatch x :

$$1. \mu, \log \sigma^2 \leftarrow E_{\phi}(x)$$

$$2. z = \mu + \exp(\frac{1}{2} \log \sigma^2) \odot \epsilon \text{ و } \epsilon \sim \mathcal{N}(0, I)$$

۳. پارامترهای likelihood را با $D_{\theta}(z)$ بسازید؛

$$4. L_{\text{rec}} = -\log p_{\theta}(x | z) \text{ و } L_{\text{KL}} \text{ را محاسبه کنید؛}$$

$$5. L = L_{\text{rec}} + \beta L_{\text{KL}}$$

۶. گرادیان، clipping اختیاری، سپس به روزرسانی مشترک θ, ϕ ؛

۷. جداگانه، KL recon، هر بعد، units active و نمونه prior را ثبت کنید.

کارگاه کد و آزمایش: VAE حداقلی

فایل `code/vae_minimal.py` یک VAE کاملاً مستقل برای بردارهای دودویی دارد و ELBO را به بازسازی و KL تفکیک می‌کند.

```
1 from __future__ import annotations
2 import torch
3 from torch import nn
4 import torch.nn.functional as F
5
6 class TinyVAE(nn.Module):
```

```

7  def __init__(self, x_dim: int = 64, z_dim: int = 8) -> None:
8      super().__init__()
9      self.enc = nn.Sequential(nn.Linear(x_dim, 48), nn.SiLU(), nn.
Linear(48, 32), nn.SiLU())
10     self.mu = nn.Linear(32, z_dim)
11     self.logvar = nn.Linear(32, z_dim)
12     self.dec = nn.Sequential(nn.Linear(z_dim, 32), nn.SiLU(), nn.
Linear(32, 48), nn.SiLU(), nn.Linear(48, x_dim))
13
14     def encode(self, x: torch.Tensor) -> tuple[torch.Tensor, torch.
Tensor]:
15         h = self.enc(x)
16         return self.mu(h), self.logvar(h).clamp(-12, 8)
17
18     @staticmethod
19     def reparameterize(mu: torch.Tensor, logvar: torch.Tensor) ->
torch.Tensor:
20         std = torch.exp(0.5 * logvar)
21         return mu + std * torch.randn_like(std)
22
23     def forward(self, x: torch.Tensor) -> tuple[torch.Tensor, torch.
Tensor, torch.Tensor]:
24         mu, logvar = self.encode(x)
25         z = self.reparameterize(mu, logvar)
26         return self.dec(z), mu, logvar
27
28     def elbo_loss(x: torch.Tensor, logits: torch.Tensor, mu: torch.Tensor,
logvar: torch.Tensor, beta: float = 1.0) -> tuple[torch.Tensor,
dict[str, float]]:
29         recon = F.binary_cross_entropy_with_logits(logits, x, reduction="
sum") / x.shape[0]
30         kl = -0.5 * torch.sum(1 + logvar - mu.square() - logvar.exp()) / x.
shape[0]
31         loss = recon + beta * kl
32         return loss, {"loss": loss.detach().item(), "recon": recon.detach
().item(), "kl": kl.detach().item()}
33
34     if __name__ == "__main__":
35         torch.manual_seed(7)
36         model = TinyVAE()
37         opt = torch.optim.Adam(model.parameters(), lr=2e-3)
38         x = torch.bernoulli(torch.full((32, 64), 0.35))
39         for _ in range(5):
40             opt.zero_grad(set_to_none=True)
41             logits, mu, logvar = model(x)
42             loss, metrics = elbo_loss(x, logits, mu, logvar)
43             loss.backward()

```

```

44     opt.step()
45     print(metrics)

```

تمرین: Bernoulli likelihood را به Gaussian با واریانس آموختنی تبدیل کنید و بررسی کنید اگر جمله $\log \sigma$ حذف شود، شبکه چگونه واریانس را بی‌قاعده افزایش می‌دهد.

۵.۱۲ فروپاشی پسین و عیب‌یابی VAE

فروپاشی پسین یعنی

$$q_{\phi}(z | x) \approx p(z) \quad (۲۳.۱۲)$$

برای تعداد زیادی از نمونه‌ها و ابعاد؛ در نتیجه z اطلاعات اندکی درباره x دارد و decoder قدرتمند تقریباً مستقل از نهفته عمل می‌کند. این پدیده فقط KL «کوچک» نیست؛ باید همراه با وابستگی بازسازی به z ، information mutual و units active بررسی شود.

۱۰.۵.۱۲ چرا رخ می‌دهد؟

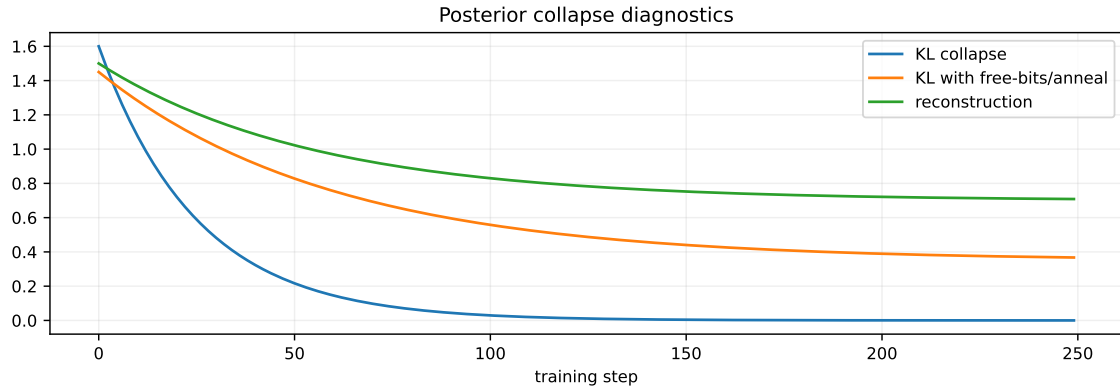
- decoder خودرگرسیو یا بسیار پر ظرفیت بدون z نیز می‌تواند داده را توضیح دهد؛
- در ابتدای آموزش، کاهش سریع KL راه ساده‌تری از استفاده از نهفته است؛
- وزن یا schedule نامناسب؛ KL
- mismatch میان prior ساده و posterior واقعی؛
- بهینه‌سازی نامتوازن encoder و decoder.

۲۰.۵.۱۲ آزمون‌های تشخیصی

۱. KL کل و KL هر بعد؛
۲. تعداد unit active با $\text{Var}_x[\mathbb{E}(z_j | x)] > \tau$ ؛
۳. بازسازی پس از shuffle کردن z ؛
۴. شکاف بین بازسازی با mean و sample؛
۵. برآورد اطلاعات متقابل $I_q(x; z)$ ؛
۶. sample prior در برابر sample posterior.

۳.۵.۱۲ راهکارها

skip decoder، weakening scheduling، capacity bits، free annealing، cyclical annealing، KL از z ، prior سلسله‌مراتبی، posterior غنی‌تر با flow و های objective جایگزین از راهکارهای رایج‌اند. هیچ کدام تضمین عمومی نیست؛ باید اثر بر نرخ، اعوجاج و کیفیت نمونه گزارش شود.



شکل ۶.۱۲: نمودار تشخیصی آموزشی. صفرشدن سریع KL در حالی که بازسازی بهبود می‌یابد می‌تواند نشانه نادیده گرفتن نهفته باشد.

۶.۱۲ خانواده گسترده VAE

۱.۶.۱۲ IWAE و کران چندنمونه‌ای

با K نمونه از q ،

$$\mathcal{L}_K = \mathbb{E} \left[\log \frac{1}{K} \sum_{k=1}^K \frac{p_\theta(x, z_k)}{q_\phi(z_k | x)} \right] \quad (۲۴.۱۲)$$

کرانی است که با افزایش K معمولاً تنگ‌تر می‌شود. با این حال، بهبود bound لزوماً به معنای نمایش بهتر encoder نیست و نسبت سیگنال به نویز گرادیان پارامترهای استنباط باید تحلیل شود.

۲.۶.۱۲ WAE و تطبیق posterior تجمعی

posterior تجمعی

$$q_\phi(z) = \int q_\phi(z | x) p_{\text{data}}(x) dx \quad (۲۵.۱۲)$$

است. WAE به جای مجبور کردن هر posterior شرطی به prior توزیع تجعی را با MMD یا منتقد خصمانه تطبیق می‌دهد:

$$\mathbb{E} c(x, G(z)) + \lambda D(q_\phi(z), p(z)). \quad (26.12)$$

این تفاوت می‌تواند کدهای اطلاعاتی تر بسازد، ولی انتخاب D و کرنل یا منتقد مهم است.

۳.۶.۱۲ مدل‌های سلسله‌مراتبی

در مدل چندسطحی

$$p(z_L) \prod_{\ell=1}^{L-1} p(z_\ell | z_{>\ell}) p(x | z_{1:L}), \quad (27.12)$$

متغیرهای سطح بالا ساختار کلی و سطوح پایین جزئیات را مدل می‌کنند. معماری‌هایی مانند NVAE و VDVAE برای جلوگیری از بی‌استفاده شدن سطوح و کنترل ناپایداری، residual، parameterization، و flow normalizing در posterior و ترفندهای بهینه‌سازی دارند. پیچیدگی این مدل‌ها نشان می‌دهد «افزودن چند latent به تنهایی سلسله‌مراتب معنایی ایجاد نمی‌کند».

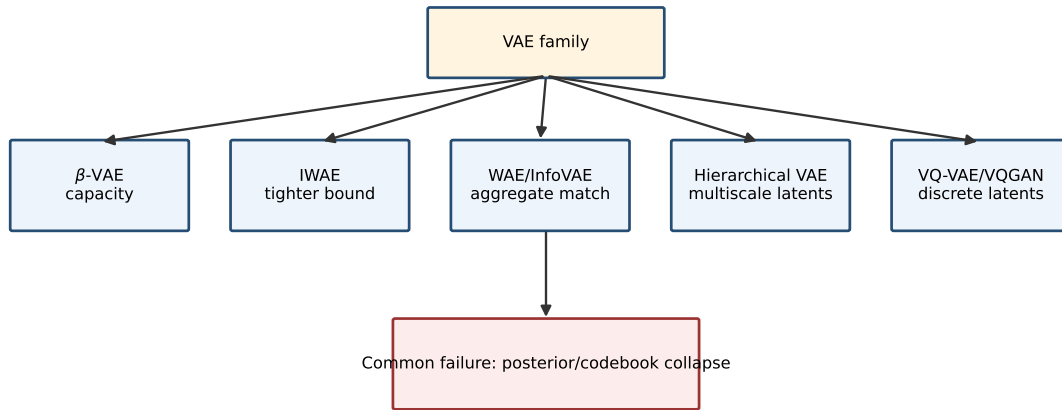
۴.۶.۱۲ VAE شرطی

برای شرط y ،

$$p_\theta(x, z | y) = p_\theta(z | y) p_\theta(x | z, y), \quad (28.12)$$

و encoder نیز $q_\phi(z | x, y)$ است. شرط می‌تواند کلاس، نقشه عمق، segmentation متن یا تصویر دیگری باشد. باید روشن شود کدام بخش شرط در encoder prior و decoder تزریق می‌شود و آیا مدل از z برای تنوع شرطی استفاده می‌کند یا خروجی تقریباً تعیین‌گراست.

VAE extensions modify the bound, prior, posterior, hierarchy, or latent alphabet



شکل ۷.۱۲: نسخه‌های VAE یک مسئله واحد را حل نمی‌کنند؛ بعضی bound، بعضی prior/posterior، بعضی هندسه نهفته و بعضی الفبای نهفته را تغییر می‌دهند.

۷.۱۲ VQ-VAE: متغیر نهفته گسسته و کد تصویری

رمزگذار بردارهای $z_e(x)$ را تولید می‌کند و هر بردار به نزدیک‌ترین عضو codebook $\{e_k\}_{k=1}^K$ نگاشت می‌شود:

$$k^* = \arg \min_k \|z_e(x) - e_k\|_2^2, \quad z_q(x) = e_{k^*}. \quad (۲۹.۱۲)$$

quantization مشتق‌پذیر نیست. estimator straight-through در backward گرادین decoder را تقریباً مستقیم به encoder می‌فرستد:

$$z_{\text{st}} = z_e + \text{sg}(z_q - z_e). \quad (۳۰.۱۲)$$

زیان رایج

$$L = L_{\text{rec}} + \|\text{sg}(z_e) - e\|_2^2 + \beta \|z_e - \text{sg}(e)\|_2^2 \quad (۳۱.۱۲)$$

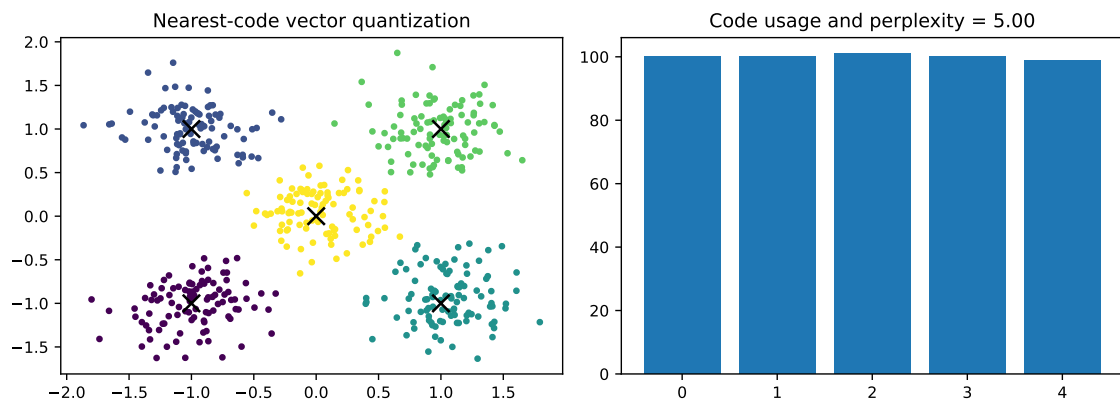
شامل به‌روزرسانی codebook و commitment است.

۱۰.۷.۱۲ Perplexity و collapse codebook

اگر p_k فراوانی استفاده از کد k باشد،

$$\text{perplexity} = \exp \left(- \sum_k p_k \log p_k \right). \quad (۳۲.۱۲)$$

perplexity پایین نسبت به K می‌تواند نشان دهد بسیاری از کدها مرده‌اند؛ ولی perplexity بالا نیز کیفیت یا معنایی بودن کد را تضمین نمی‌کند. باید، reconstruction، entropy rFID، occupancy، assignment و بعد مؤثر codebook با هم گزارش شوند.



شکل ۸.۱۲: کوانتیزه‌سازی برداری و سنجش استفاده از codebook. فروپاشی ممکن است به چند کد فعال یا زیر فضای بسیار کم بعد محدود شود.

۲۰۷۰۱۲ به‌روزرسانی codebook EMA

به‌جای گرادینان مستقیم می‌توان شمارش و مجموع بردارهای هر کد را با میانگین نمایی به‌روزرسانی کرد. این روش شبیه k-means آنلاین است و گاهی پایداری بیشتری دارد. initialization با reset k-means، کدهای مرده، residual، quantization product quantization و روش‌های بدون codebook مانند quantization scalar finite نیز ابزارهای طراحی‌اند.

۳۰۷۰۱۲ VQGAN و حد بالای کیفیت tokenizer

VQGAN بازسازی را با زبان ادراکی و منتقد patch ترکیب می‌کند تا grid latent فشرده ولی از نظر ادراکی دقیق بسازد. سپس یک Transformer توزیع کدها را مدل می‌کند. این جداسازی دو خطا ایجاد می‌کند:

$$\text{خطای prior روی ها} + \text{خطای tokenizer} = \text{خطای نهایی} \quad (۳۳.۱۲)$$

اگر tokenizer جزئیات را حذف کند، مدل مولد مرحله دوم قادر به بازیابی وفادار آن‌ها نیست؛ ممکن است صرفاً جزئیات plausible بسازد.

کارگاه کد و آزمایش: VQ-VAE حداقلی

فایل code/vq_vae_minimal.py، nearest-code، loss commitment straight-through و perplexity را پیاده می‌کند.

```
1 from __future__ import annotations
```

```

2 import torch
3 from torch import nn
4 import torch.nn.functional as F
5
6 class VectorQuantizer(nn.Module):
7     def __init__(self, codes: int = 32, dim: int = 8, beta: float =
8         0.25) -> None:
9         super().__init__()
10        self.codebook = nn.Embedding(codes, dim)
11        nn.init.uniform_(self.codebook.weight, -1 / codes, 1 / codes)
12        self.beta = beta
13
14    def forward(self, z_e: torch.Tensor) -> tuple[torch.Tensor, torch.
15        Tensor, torch.Tensor, torch.Tensor]:
16        flat = z_e.reshape(-1, z_e.shape[-1])
17        dist = flat.square().sum(1, keepdim=True) - 2 * flat @ self.
18        codebook.weight.T + self.codebook.weight.square().sum(1)
19        ids = dist.argmax(1)
20        z_q = self.codebook(ids).view_as(z_e)
21        codebook_loss = F.mse_loss(z_q, z_e.detach())
22        commit_loss = F.mse_loss(z_e, z_q.detach())
23        loss = codebook_loss + self.beta * commit_loss
24        z_st = z_e + (z_q - z_e).detach()
25        probs = torch.bincount(ids, minlength=self.codebook.
26        num_embeddings).float()
27        probs = probs / probs.sum().clamp_min(1)
28        perplexity = torch.exp(-(probs * (probs + 1e-12).log()).sum())
29        return z_st, loss, ids.view(z_e.shape[:-1]), perplexity
30
31 class TinyVQVAE(nn.Module):
32     def __init__(self) -> None:
33         super().__init__()
34         self.enc = nn.Sequential(nn.Linear(64, 48), nn.SiLU(), nn.
35         Linear(48, 4 * 8))
36         self.vq = VectorQuantizer(32, 8)
37         self.dec = nn.Sequential(nn.Linear(4 * 8, 48), nn.SiLU(), nn.
38         Linear(48, 64))
39
40    def forward(self, x: torch.Tensor):
41        z_e = self.enc(x).view(x.shape[0], 4, 8)
42        z_q, qloss, ids, perplexity = self.vq(z_e)
43        logits = self.dec(z_q.flatten(1))
44        return logits, qloss, ids, perplexity
45
46 if __name__ == "__main__":
47     torch.manual_seed(3)
48     model = TinyVQVAE()

```

```

43 opt = torch.optim.Adam(model.parameters(), lr=2e-3)
44 x = torch.rand(24, 64)
45 for _ in range(5):
46     opt.zero_grad(set_to_none=True)
47     logits, qloss, ids, perplexity = model(x)
48     loss = F.mse_loss(torch.sigmoid(logits), x) + qloss
49     loss.backward(); opt.step()
50 print({"loss": loss.detach().item(), "perplexity": perplexity.
    detach().item(), "used_codes": int(ids.unique().numel())})

```

آزمایش پیشنهادی: K ، بعد code و β را تغییر دهید و سه نمودار، perplexity reconstruction و codebook rank effective را هم‌زمان رسم کنید.

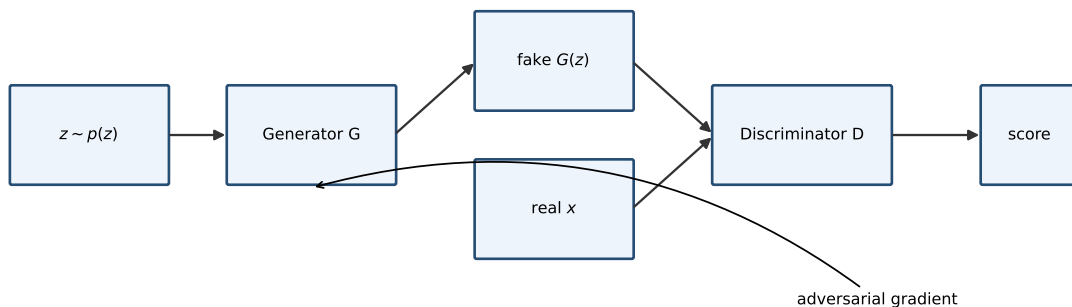
۸.۱۲ GAN: یادگیری توزیع به‌وسیله بازی تمایز

مدل اصلی GAN بازی زیر را تعریف می‌کند:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}} \log D(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D(G(z))). \quad (۳۴.۱۲)$$

$D(x)$ احتمال واقعی بودن نیست مگر تحت فرض‌های دقیق و کالیبراسیون مناسب؛ در تحلیل نظری تمایزگر بهینه، یک نسبت چگالی است.

A GAN is a coupled optimization game, not a single minimization problem



شکل ۹.۱۲: حلقه GAN. تمایزگر تابع زیان داده‌محور برای مولد می‌سازد و مولد هم‌زمان توزیع ورودی تمایزگر را تغییر می‌دهد.

۱۰.۸.۱۲ استخراج تمایزگر بهینه

برای G ثابت، تابع هدف نقطه به نقطه

$$p_{\text{data}}(x) \log D(x) + p_g(x) \log(1 - D(x)) \quad (35.12)$$

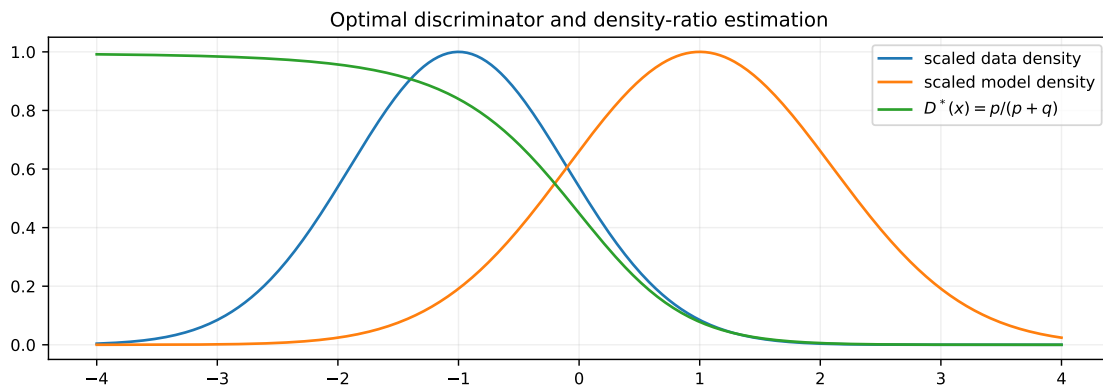
است. مشتق نسبت به $D(x)$ را صفر می‌کنیم:

$$\frac{p_{\text{data}}}{D} - \frac{p_g}{1 - D} = 0 \implies D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}. \quad (36.12)$$

جایگذاری می‌دهد:

$$V(D^*, G) = -\log 4 + 2 \text{JS}(p_{\text{data}} \| p_g). \quad (37.12)$$

این نتیجه درباره تمایزگر بهینه و ظرفیت نامحدود است؛ آموزش عملی با شبکه محدود، minibatch و بهینه‌سازی ناقص دقیقاً این واگرایی را کمینه نمی‌کند.



شکل ۱۰.۸.۱۲: تمایزگر بهینه نسبت چگالی را به یک احتمال لوجستیک تبدیل می‌کند. در نواحی فقط واقعی به یک و فقط مصنوعی به صفر میل می‌کند.

۲۰.۸.۱۲ زیان غیر اشباع مولد

در نسخه minimax، هنگامی که $D(G(z)) \approx 0$ ، gradient از sigmoid ممکن است ضعیف باشد. زیان غیر اشباع

$$L_G^{\text{NS}} = -\mathbb{E}_z \log D(G(z)) \quad (38.12)$$

همان نقطه تعادل را هدف می‌گیرد ولی gradient قوی‌تری در ابتدای آموزش دارد. loss hinge نیز در مدل‌های تصویری رایج است:

$$L_D = \mathbb{E}_x \max(\cdot, 1 - D(x)) + \mathbb{E}_z \max(\cdot, 1 + D(G(z))), \quad (39.12)$$

$$L_G = -\mathbb{E}_z D(G(z)). \quad (40.12)$$

۹.۱۲ چرا GAN ناپایدار است؟

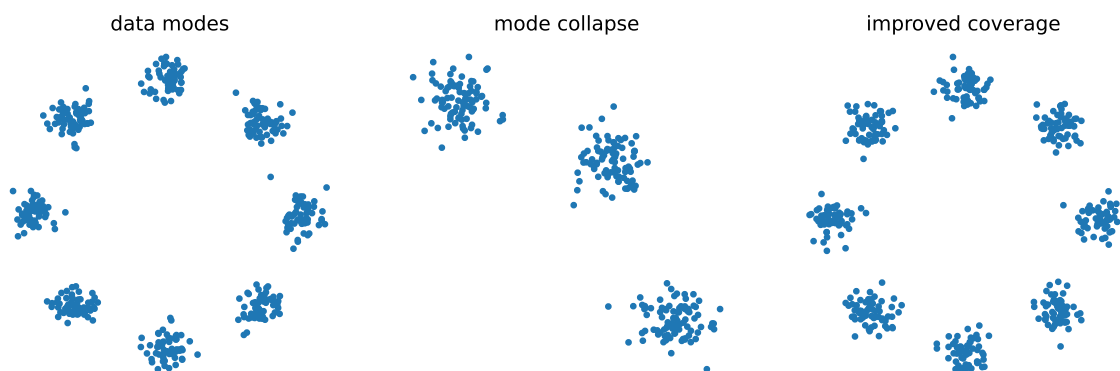
آموزش GAN کینه‌سازی یک تابع ثابت نیست، بلکه یافتن تعادل یک بازی است. اگر پارامترها θ_G, θ_D باشند، میدان گرادیان

$$F(\theta) = \begin{bmatrix} \nabla_{\theta_G} L_G \\ \nabla_{\theta_D} L_D \end{bmatrix} \quad (41.12)$$

می‌تواند مؤلفه دورانی داشته باشد. بنابراین loss ممکن است نوسان کند و کاهش یکنواخت انتظار درستی نیست.

۱۰.۹.۱۲ فروپاشی مد

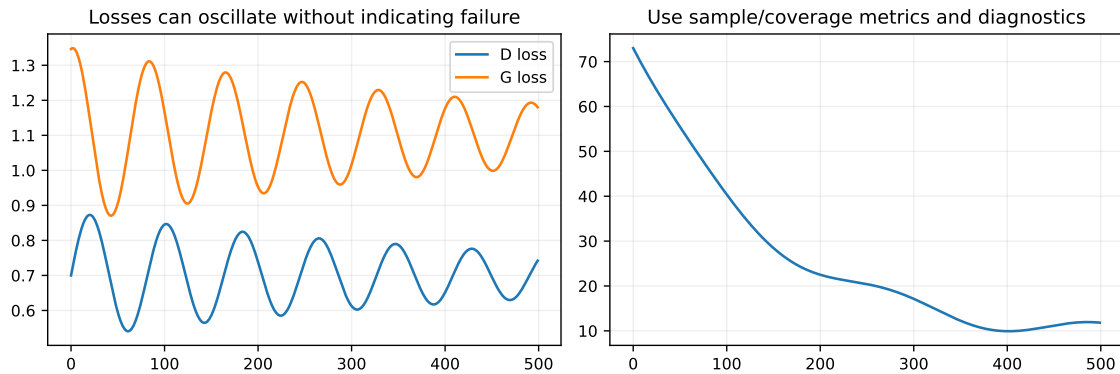
مولد ممکن است چندین z را به یک یا چند مد محدود ببرد. علت‌ها شامل gradient مشابه، منتقد ناکافی، عدم تعادل سرعت یادگیری، هدف mode-seeking و داده محدود است. تشخیص باید با pre-coverage، occupancy cision/recall، nearest-neighbor diversity، pairwise و آزمون sensi-latent tivity انجام شود؛ تصویر خوب منتخب، collapse را رد نمی‌کند.



شکل ۱۱.۱۲: نمایش دوبعدی فروپاشی مد. کیفیت موضعی نمونه‌ها ممکن است بالا باشد، ولی پوشش توزیع ناقص بماند.

۲۰۹۰۱۲ برتری بیش از حد تمایزگر

تمایزگر بسیار قوی می‌تواند داده آموزش را حفظ کند و سطح تصمیم تیز بسازد؛ در نتیجه gradient مولد بد شرط می‌شود. تمایزگر بسیار ضعیف نیز سیگنال مفیدی نمی‌دهد. نسبت تعداد، ها، update، ظرفیت، augmentation، regularization و نرخ یادگیری باید بر اساس تشخیص، نه قاعده ثابت، تنظیم شود.



شکل ۱۲۰۱۲: زیان‌های GAN به تنهایی معیار کیفیت نیستند. نمونه ثابت، معیار پوشش، norm گرادیان و FID/KID روی پروتکل ثابت لازم‌اند.

۱۰۰۱۲ WGAN و هندسه Lipschitz

WGAN از دوگان Wasserstein استفاده می‌کند:

$$\max_{\psi: \|f_{\psi}\|_L \leq 1} \mathbb{E}_{x \sim p_{\text{data}}} f_{\psi}(x) - \mathbb{E}_z f_{\psi}(G_{\theta}(z)). \quad (۴۲۰۱۲)$$

خروجی f دیگر احتمال نیست و critic نامیده می‌شود. قید یک-Lipschitz بخش اصلی نظریه است.

۱۰۱۰۱۲ چرا clipping weight نامناسب است؟

clipping فضای پارامتر را محدود می‌کند، نه مستقیماً ثابت Lipschitz تابع را. می‌تواند ظرفیت critic را کم و gradient را اشباع یا قطعه‌ای کند.

۲۰۱۰۱۲ penalty Gradient

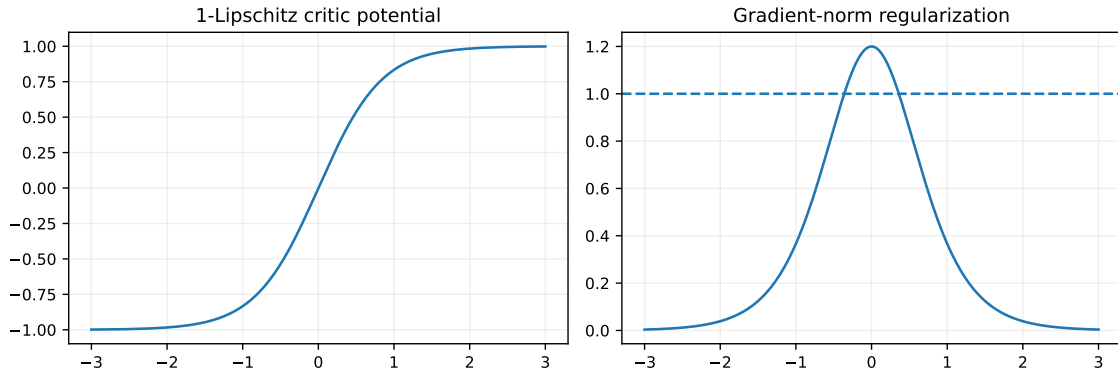
WGAN-GP روی نقاط میان داده واقعی و مصنوعی

$$\hat{x} = \epsilon x + (1 - \epsilon)\tilde{x}, \quad \epsilon \sim U(0, 1) \quad (۴۳۰۱۲)$$

جریمه

$$\lambda \mathbb{E}_{\hat{x}} (\|\nabla_{\hat{x}} f(\hat{x})\|_2 - 1)^2 \quad (۴۴.۱۲)$$

را اضافه می‌کند. این قید تنها روی مسیرهای نمونه‌برداری شده اعمال می‌شود و تضمین Lipschitz global کامل نیست.



شکل ۱۳.۱۲: critic هموار و جریمه norm گرادیان. هدف، فراهم کردن شیب معنادار میان تکیه‌گاه‌های داده و مدل است.

۳.۱۰.۱۲ Spectral normalization

برای لایه خطی $h = Wx$ ، ثابت Lipschitz با $\sigma_{\max}(W)$ کران می‌شود. Spectral normalization از

$$\bar{W} = \frac{W}{\sigma_{\max}(W)} \quad (۴۵.۱۲)$$

استفاده می‌کند و بزرگ‌ترین مقدار منفرد را با iteration power تقریب می‌زند. حاصل ضرب کران لایه‌ها یک کران شبکه می‌دهد، هرچند شل است.

۴.۱۰.۱۲ lazy regularization و TTUR

جریمه $\mathcal{R}$

$$\frac{\gamma}{2} \mathbb{E}_{x \sim p_{\text{data}}} \|\nabla_x D(x)\|^2 \quad (۴۶.۱۲)$$

تمایزگر را نزدیک داده واقعی هموار می‌کند. در StyleGAN این جریمه ممکن است هر چند گام یک‌بار اعمال و ضریب آن برای حفظ امید تنظیم شود. TTUR نرخ‌های یادگیری متفاوت برای دو بازیکن دارد و EMA وزن‌های مولد برای نمونه‌گیری پایدار استفاده می‌شود.

الگوریتم ۲۰۱۲ - یک گام WGAN-GP

۱. $\tilde{x} = G(z)$ و $z \sim p(z)$ ، $x \sim p_{\text{data}}$

۲. $\mathbb{E}f(\tilde{x}) - \mathbb{E}f(x) + \lambda GP \text{ loss: critic}$

۳. چند critic update برای هر update مولد، در صورت نیاز؛

۴. مولد: $L_G = -\mathbb{E}f(G(z))$

۵. norm گرادیان، score واقعی/مصنوعی، coverage و نمونه‌های ثابت ثبت شوند؛

۶. EMA مولد فقط برای ارزیابی نگهداری شود و وزن online نیز ذخیره گردد.

کارگاه کد و آزمایش: WGAN-GP و GAN روی مخلوط گاوسی

```

1 from __future__ import annotations
2 import torch
3 from torch import nn
4 import torch.nn.functional as F
5
6 class Generator(nn.Module):
7     def __init__(self, z_dim: int = 8) -> None:
8         super().__init__()
9         self.net = nn.Sequential(nn.Linear(z_dim, 32), nn.LeakyReLU
10         (0.2), nn.Linear(32, 32), nn.LeakyReLU(0.2), nn.Linear(32, 2))
11     def forward(self, z: torch.Tensor) -> torch.Tensor: return self.
12     net(z)
13
14 class Discriminator(nn.Module):
15     def __init__(self) -> None:
16         super().__init__()
17         self.net = nn.Sequential(nn.Linear(2, 32), nn.LeakyReLU(0.2),
18         nn.Linear(32, 32), nn.LeakyReLU(0.2), nn.Linear(32, 1))
19     def forward(self, x: torch.Tensor) -> torch.Tensor: return self.
20     net(x).squeeze(-1)
21
22 def sample_real(n: int) -> torch.Tensor:
23     k = torch.randint(0, 8, (n,))
24     angle = 2 * torch.pi * k.float() / 8
25     centers = torch.stack((2 * torch.cos(angle), 2 * torch.sin(angle)),
26     , 1)
27     return centers + 0.08 * torch.randn(n, 2)
28
29 if __name__ == "__main__":
30     torch.manual_seed(1)
31     G, D = Generator(), Discriminator()

```

```

27 og = torch.optim.Adam(G.parameters(), 2e-4, betas=(0.5, 0.9)); od
   = torch.optim.Adam(D.parameters(), 2e-4, betas=(0.5, 0.9))
28 for _ in range(20):
29     real = sample_real(128); fake = G(torch.randn(128, 8)).detach
   ()
30     od.zero_grad(set_to_none=True)
31     dloss = F.softplus(-D(real)).mean() + F.softplus(D(fake)).mean
   (); dloss.backward(); od.step()
32     og.zero_grad(set_to_none=True)
33     fake = G(torch.randn(128, 8)); gloss = F.softplus(-D(fake)).
   mean(); gloss.backward(); og.step()
34     print({"d_loss": dloss.detach().item(), "g_loss": gloss.detach().
   item(), "sample_std": G(torch.randn(512, 8)).std(0).detach().tolist
   ()})

```

فایل `code/wgan_gp_minimal.py` جریه گرایان را با autograd محاسبه می‌کند. روی هشت مد دوبعدی، occupancy هر مد را در طول آموزش رسم کنید و نشان دهید loss تنها برای تشخیص collapse کافی نیست.

۱۱.۱۲ معماری‌های مهم GAN

۱۰۱۱.۱۲ DCGAN: قرارداد معماری پایه

DCGAN استفاده منظم از convolution با stride، batch normalization، ReLU در مولد و LeakyReLU در تمایزگر را تثبیت کرد. اهمیت آن بیشتر از یک مدل تاریخی است؛ یک baseline شفاف برای بررسی تابع هدف و خط لوله داده است. با این حال، transposed convolution می‌تواند الگوی شطرنجی بسازد و BatchNorm در دسته‌های کوچک ناپایدار است.

۲۰۱۱.۱۲ discriminator projection و شرطی سازی

در GAN شرطی، $G(z, y)$ و $D(x, y)$ داریم. discriminator projection score را به صورت

$$D(x, y) = u^T h(x) + e(y)^T h(x) \quad (۴۷.۱۲)$$

می‌سازد و سازگاری ویژگی تصویر با embedding کلاس را مستقیماً می‌سنجد. ACGAN یک سر طبقه‌بندی نیز اضافه می‌کند، ولی ممکن است کیفیت و تنوع را با هدف طبقه‌بندی درهم پیامیزد.

۳.۱۱.۱۲ PatchGAN و pix2pix

برای داده زوج شده (x, y) ،

$$L = L_{\text{cGAN}}(G, D) + \lambda \|y - G(x)\|_1. \quad (۴۸.۱۲)$$

منتقد patch به جای یک score کلی، واقعی بودن وصله‌ها را می‌سنجد؛ بنابراین جزئیات بافت را تقویت می‌کند ولی سازگاری جهانی را تضمین نمی‌کند.

۴.۱۱.۱۲ CycleGAN و عدم شناساپذیری

برای داده بدون زوج، دو نگاشت $G: X \rightarrow Y$ و $F: Y \rightarrow X$ و زیان

$$L_{\text{cyc}} = \mathbb{E}_x \|F(G(x)) - x\|_1 + \mathbb{E}_y \|G(F(y)) - y\|_1 \quad (۴۹.۱۲)$$

اضافه می‌شود. consistency cycle نگاشت را محدود می‌کند ولی correspondence معنایی یگانه‌سازی سازد؛ نگاشت‌های اطلاعات پنهان و تغییرات نامطلوب ممکن‌اند. برای کاربرد پزشکی یا علمی، حفظ ساختار باید با قیدهای مستقل آزمون شود.

۵.۱۱.۱۲ BigGAN و مقیاس

BigGAN نشان داد batch بزرگ، مدل بزرگ، class، normalization spectral embedding و truncation می‌توانند کیفیت شرطی را افزایش دهند. truncation z را به ناحیه پرچگالی prior محدود می‌کند و معمولاً precision را به بهای recall افزایش می‌دهد؛ بنابراین یک knob ارزیابی است، نه بهبود رایگان.

۱۲.۱۲ خانواده StyleGAN و هندسه فضای نهفته

StyleGAN نگاشت $z \mapsto w$ را از synthesis جدا و style را در هر مقیاس تزریق می‌کند. فضای‌های متداول:

• $\mathcal{Z}$: prior ورودی؛

• $\mathcal{W}$: خروجی mapping network؛

• $\mathcal{W}^+$: style مستقل برای هر لایه؛

• StyleSpace: پارامترهای modulation لایه‌ها.

افزایش آزادی inversion را آسان‌تر ولی prior مولد را ضعیف‌تر می‌کند؛ projection دقیق در W^+ الزاماً نمونه طبیعی روی منیفلد مدل نیست.

۱۰۱۲.۰۱۲ Modulation و demodulation در StyleGAN ۲

وزن convolution برای نمونه با style s_i مدوله می‌شود:

$$w'_{ijk} = s_i w_{ijk}, \quad (۵۰.۱۲)$$

سپس برای کنترل مقیاس خروجی demodulate می‌شود:

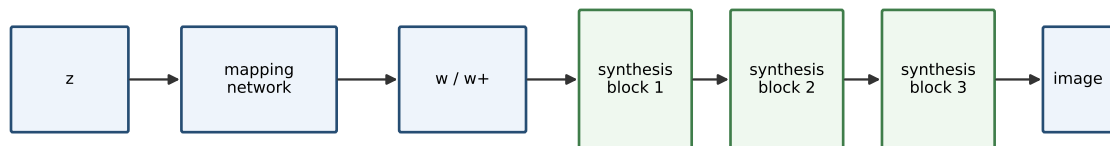
$$w''_{ijk} = \frac{w'_{ijk}}{\sqrt{\sum_{i,k} (w'_{ijk})^2 + \epsilon}}. \quad (۵۱.۱۲)$$

regularization path-length تلاش می‌کند تغییر هم‌اندازه در w اثر قابل پیش‌بینی‌تری در تصویر داشته باشد.

۲۰۱۲.۰۱۲ alias و حذف StyleGAN ۳

StyleGAN ۳ سیگال‌های داخلی را نمونه‌هایی از میدان پیوسته در نظر می‌گیرد و resampling فیلترشده و عملیات equivariant را برای جلوگیری از sticking texture به کار می‌برد. این دیدگاه، فصل نمونه‌برداری و aliasing را مستقیماً به طراحی مولد عمیق پیوند می‌دهد.

Style-based synthesis separates latent mapping from spatial synthesis



style modulation + noise + controlled resampling

شکل ۱۴.۱۲: ساختار مفهومی StyleGAN. network mapping هندسه prior را تغییر می‌دهد و های style چندمقیاسی synthesis را کنترل می‌کنند.

۳۰۱۲.۰۱۲ Inversion و ویرایش

Inversion می‌تواند با بهینه‌سازی encoder latent، یا روش هیبرید انجام شود:

$$w^* = \arg \min_w \ell_{\text{pix}}(G(w), x) + \lambda \ell_{\text{perc}}(G(w), x) + R(w). \quad (۵۲.۱۲)$$

سه معیار جدا لازم است: fidelity بازسازی، editability و قرارداشتن در ناحیه طبیعی. latent بهبود یکی ممکن است دیگری را تضعیف کند.

۱۳.۱۲ مدل‌های خودرگرسیو: چگالی دقیق با قاعده زنجیره

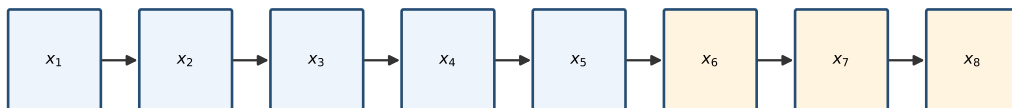
برای ترتیب ثابت مؤلفه‌ها،

$$p(x) = \prod_{i=1}^D p(x_i | x_{<i}). \quad (۵۳.۱۲)$$

در آموزش همه حالت‌ها با forcing teacher به‌طور موازی محاسبه می‌شوند، اما نمونه‌گیری معمولاً ترتیبی است.

Autoregressive factorization: exact likelihood, sequential sampling

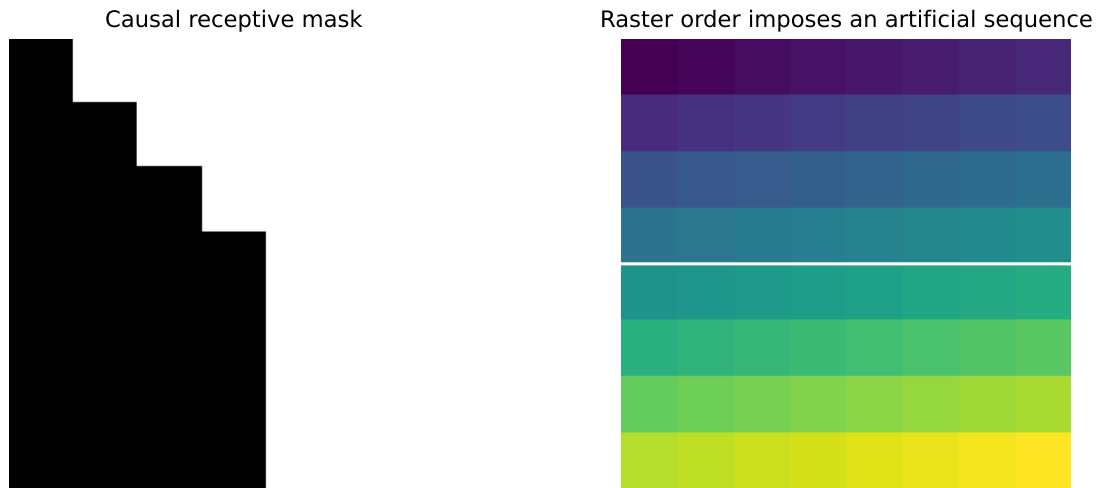
$$p(x) = \prod_i p(x_i | x_{<i})$$



شکل ۱۵.۱۲: قاعده زنجیره، likelihood دقیق را به‌های conditional ساده‌تر می‌شکند؛ هزینه آن ترتیب مصنوعی و sampling متوالی است.

۱۰.۱۳.۱۲ PixelCNN و convolution masked

mask باید تضمین کند خروجی مکان i به x_i یا آینده دسترسی ندارد. ماسک نوع A در لایه نخست مرکز را نیز حذف می‌کند و نوع B در لایه‌های بعدی مرکز feature قبلی را مجاز می‌داند. خطای یک پیکسل در mask باعث leakage و درست‌نمایی جعلی می‌شود.



شکل ۱۲.۱۶: mask علی و ترتیب raster. ترتیب پیکسل یک ساختار فیزیکی ذاتی تصویر نیست و روی دشواری مدل اثر می‌گذارد.

۲.۱۳.۱۲ likelihood پیکسل رنگی

مدل کردن هر مقدار ۸ بیتی با softmax ۲۵۶ تایی ممکن است وابستگی کانال‌ها را نادیده بگیرد. discretized mean mixture، logits of mixture جرم بازه کوانتیزه را از اختلاف CDF محاسبه می‌کند و پارامترهای و scale را می‌آموزد. گزارش bits/dim باید دقیقاً با قرارداد dequantization و ordering channel همراه باشد.

کارگاه کد و آزمایش: آزمون علیت PixelCNN

فایل code/pixelcnn_mask.py یک masked convolution آموزش می‌دهد. آزمون همراه آخرین پیکسل را تغییر می‌دهد و بررسی می‌کند logits مکان‌های قبلی تغییر نکنند.

```

1 from __future__ import annotations
2 import torch
3 from torch import nn
4 import torch.nn.functional as F
5
6 class MaskedConv2d(nn.Conv2d):
7     def __init__(self, mask_type: str, *args, **kwargs):
8         super().__init__(*args, **kwargs)
9         if mask_type not in {"A", "B"}: raise ValueError("mask_type
10 must be A or B")
11         mask = torch.ones_like(self.weight)
12         h,w = self.kernel_size; yc,xc=h//2,w//2
13         mask[:, :, yc+1:, :] = 0; mask[:, :, yc, xc+(mask_type=="B"):] = 0
14         self.register_buffer("mask", mask)
15     def forward(self, x): return F.conv2d(x, self.weight*self.mask, self.
16 bias, self.stride, self.padding, self.dilation, self.groups)

```

```

15
16 class TinyPixelCNN(nn.Module):
17     def __init__(self, levels: int=16):
18         super().__init__(); self.net=nn.Sequential(MaskedConv2d('A'
19 ,1,32,7,padding=3),nn.ReLU(),MaskedConv2d('B',32,32,3,padding=1),nn
20 .ReLU(),nn.Conv2d(32,levels,1))
21     def forward(self,x): return self.net(x)
22
23 if __name__=='__main__':
24     torch.manual_seed(5); model=TinyPixelCNN(); x=torch.rand(2,1,8,8);
25     y=model(x)
26     x2=x.clone(); x2[:, :, 7, 7]=1-x2[:, :, 7, 7]
27     delta=(model(x2)[ :, :, :7, :]-y[ :, :, :7, :]).abs().max()
28     print({"output_shape": list(y.shape), "causality_test_max_delta":
29 delta.detach().item()})

```

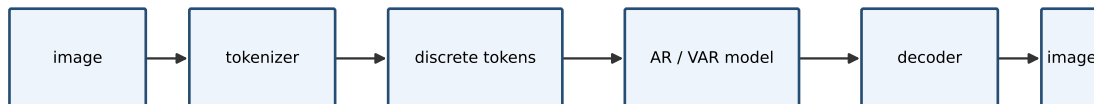
این آزمون test unit مهم‌تری از مشاهده ظاهری نمونه‌هاست.

۱۴.۱۲ از پیکسل به توکن: مولدهای تصویری خودرگرسیو جدید

نمونه‌گیری پیکسلی برای تصویر بزرگ بسیار طولانی است. راهکار غالب، آموختن tokenizer و مدل کردن دنباله کدهاست:

$$x \xrightarrow{E,Q} c_{1:N} \xrightarrow{p_\psi(c)} \hat{c}_{1:N} \xrightarrow{D} \hat{x}. \quad (۵۴.۱۲)$$

Two-stage visual generation with learned discrete tokens



Tokenizer quality sets the upper bound; sequence design sets generation cost.

شکل ۱۷.۱۲: مولد دومرحله‌ای. کیفیت tokenizer حد بالای وفاداری و تعداد/ترتیب token هزینه prior را تعیین می‌کند.

۱۰۱۴۰۱۲ Transformer + VQGAN

کاهش تصویر به grid latent امکان مدل کردن ترکیب‌های دوربرد با Transformer را فراهم می‌کند. attention sliding برای رزولوشن بزرگ استفاده می‌شود. در وظایف شرطی، های token کلاس، متن، segmentation یا edge به دنباله افزوده می‌شوند.

۲۰۱۴۰۱۲ MaskGIT و decoding موازی تکراری

به جای ترتیب چپ‌به‌راست، مدل های token شده mask را پیش‌بینی می‌کند و در چند دور، مطمئن‌ترین ها را ثابت می‌سازد. این روش sampling را از N گام به تعداد دورهای کم کاهش می‌دهد، ولی independence conditional تقریبی و mask schedule مهم‌اند.

۳۰۱۴۰۱۲ LlamaGen و prediction next-token

LlamaGen نشان می‌دهد یک Transformer علی استاندارد روی های token تصویری، با tokenizer و scaling مناسب، می‌تواند مولد رقابتی باشد. مزیت مهندسی آن استفاده از زیرساخت serving مدل‌های زبانی است؛ محدودیت اصلی طول دنباله، خطای تجمعی و ترتیب raster است.

۴۰۱۴۰۱۲ VAR و prediction next-scale

VAR به جای token بعدی، نقشه token مقیاس بعدی را از coarse به fine پیش‌بینی می‌کند:

$$p(r_1, \dots, r_S) = \prod_{s=1}^S p(r_s | r_{<s}). \quad (۵۵۰۱۲)$$

این ترتیب با ساختار چندمقیاسی تصویر سازگارتر و تعداد مراحل کمتر است. با این حال، ادعای برتری باید روی tokenizer، داده، compute و معیار یکسان سنجیده شود.

۵۰۱۴۰۱۲ توکن‌های یک بعدی فشرده

TiTok تصویر را به دنباله کوتاه D_1 تبدیل می‌کند. فشردگی بسیار بالا هزینه مولد را کاهش می‌دهد، اما فشار بیشتری به decoder و compression semantic وارد می‌کند. پژوهش‌های ۲۰۲۵-۲۰۲۶ به length token تطبیقی، های token مبتنی بر model foundation و جلوگیری از collapse dimensional از codebook پرداخته‌اند.

اصل مهندسی مولد توکنی

هر نتیجه generation باید همراه با نتیجه مستقل tokenizer گزارش شود: FID، reconstruction، LPIPS/PSNR، استفاده، codebook طول توکن، سرعت encoder/decoder و حداکثر رزولوشن.

در غیر این صورت معلوم نیست بهبود از prior است یا tokenizer.

۱۵.۱۲ جریان‌های نرمال‌ساز: چگالی دقیق و نگاشت وارون‌پذیر

اگر $f_\theta : x \mapsto z$ دوسویی و مشتق‌پذیر باشد،

$$\log p_X(x) = \log p_Z(f_\theta(x)) + \log \left| \det \frac{\partial f_\theta(x)}{\partial x} \right|. \quad (۵۶.۱۲)$$

نمونه‌گیری با $z \sim p_Z$ و $x = f_\theta^{-1}(z)$ انجام می‌شود. چالش، ساخت تبدیل بیانگر با determinant ژاکوبین ارزان است.

۱۰۱۵.۱۲ RealNVP در coupling Affine

$x = (x_a, x_b)$ را تقسیم کنید:

$$y_a = x_a, \quad (۵۷.۱۲)$$

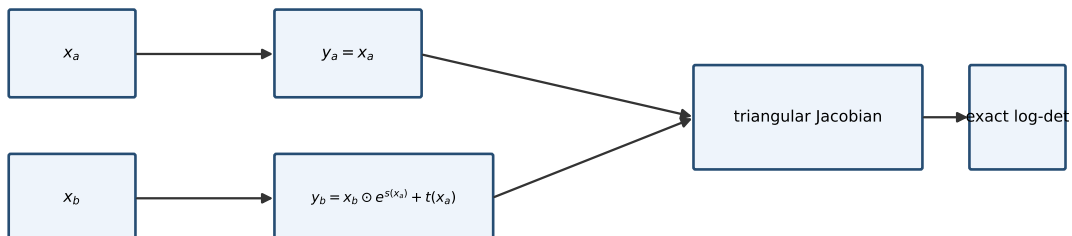
$$y_b = x_b \odot \exp s(x_a) + t(x_a). \quad (۵۸.۱۲)$$

ژاکوبین مثلثی است و

$$\log |\det J| = \sum_j s_j(x_a). \quad (۵۹.۱۲)$$

وارون نیز مستقیم است. نیی از متغیرها در هر لایه ثابت می‌مانند؛ permutation یا convolution invertible برای اختلاط لازم است.

Affine coupling trades unrestricted transforms for tractable Jacobians



شکل ۱۸.۱۲: coupling وارون‌پذیر. ساختار مثلثی ژاکوبین محاسبه log-determinant را ارزان می‌کند.

Glow ۲.۱۵.۱۲

Glow سه جزء را تکرار می‌کند: convolution و 1×1 و Act- coupling, affine و Norm آمار اولیه داده را برای تنظیم bias/scale استفاده می‌کند؛ convolution و 1×1 و permutation factor-out می‌کند. معماری multi-scale بخشی از متغیرها را در سطوح مختلف factor-out می‌کند.

IAF و MAF ۳.۱۵.۱۲

MAF محاسبه چگالی را موازی و نمونه‌گیری را ترتیبی می‌کند؛ IAF برعکس، نمونه‌گیری موازی و محاسبه density معکوس را مناسب‌تر برای posterior واریاسیونی می‌سازد. انتخاب جهت باید با وظیفه inference یا generation هماهنگ باشد.

Dequantization ۴.۱۵.۱۲

پیکسل گسسته را نمی‌توان بی‌واسطه با چگالی پیوسته مقایسه کرد. dequantization uniform

$$y = x + u, \quad u \sim U[0, 1) \quad (۶۰۰۱۲)$$

یک bound روی جرم گسسته می‌سازد. dequantization variational توزیع نویز شرطی را می‌آموزد. bits/dim بدون ذکر روش dequantization قابل مقایسه نیست.

کارگاه کد و آزمایش: RealNVP دوبعدی و آزمون وارون

```

1 from __future__ import annotations
2 import math
3 import torch
4 from torch import nn
5
6 class Coupling(nn.Module):
7     def __init__(self, flip: bool = False) -> None:
8         super().__init__(); self.flip = flip
9         self.st = nn.Sequential(nn.Linear(1,32), nn.Tanh(), nn.Linear
10 (32,32), nn.Tanh(), nn.Linear(32,2))
11     def forward(self, x: torch.Tensor) -> tuple[torch.Tensor, torch.
12 Tensor]:
13         a,b = (x[:,1:2],x[:,0:1]) if self.flip else (x[:,0:1],x
14[:,1:2])
15         s,t = self.st(a).chunk(2,1); s = .8*torch.tanh(s)
16         yb = b*torch.exp(s)+t
17         y = torch.cat((yb,a),1) if self.flip else torch.cat((a,yb),1)
18         return y, s.squeeze(1)
19     def inverse(self, y: torch.Tensor) -> torch.Tensor:

```

```

17     a,b = (y[:,1:2],y[:,0:1]) if self.flip else (y[:,0:1],y
18    [:,1:2])
19     s,t = self.st(a).chunk(2,1); s=.8*tanh(s)
20     xb=(b-t)*torch.exp(-s)
21     return torch.cat((xb,a),1) if self.flip else torch.cat((a,xb)
22     ,1)
23
24 class Flow(nn.Module):
25     def __init__(self): super().__init__(); self.layers=nn.ModuleList
26     ([Coupling(False),Coupling(True),Coupling(False),Coupling(True)])
27     def forward(self,x):
28         logdet=torch.zeros(x.shape[0]); z=x
29         for layer in self.layers:
30             z,ld=layer(z); logdet=logdet+ld
31         return z,logdet
32     def inverse(self,z):
33         x=z
34         for layer in reversed(self.layers): x=layer.inverse(x)
35         return x
36     def nll(self,x):
37         z,ld=self(x); logpz=-.5*(z.square().sum(1)+2*math.log(2*math.
38         pi)); return -(logpz+ld).mean()
39
40 if __name__=='__main__':
41     torch.manual_seed(4); f=Flow(); x=torch.randn(64,2)
42     z,ld=f(x); xr=f.inverse(z)
43     print({"reconstruction_max_error": (x-xr).abs().max().detach().
44     item(), "mean_logdet": ld.mean().detach().item(), "nll": f.nll(x).
45     detach().item()})

```

آزمون اصلی باید $\max |x - f^{-1}(f(x))|$ را نزدیک دقت ماشین نشان دهد. سپس log-det تحلیل را با determinant عددی ژاکوبین در بعد کوچک مقایسه کنید.

۱۶.۱۲ جریان‌های پیوسته و Matching Flow

در normalizing continuous flow، حالت از ODE پیروی می‌کند:

$$\frac{dx_t}{dt} = v_\theta(x_t, t). \quad (۶۱.۱۲)$$

تغییر log-density با معادله آنی

$$\frac{d}{dt} \log p_t(x_t) = -\operatorname{tr} \left(\frac{\partial v_\theta}{\partial x_t} \right) \quad (۶۲.۱۲)$$

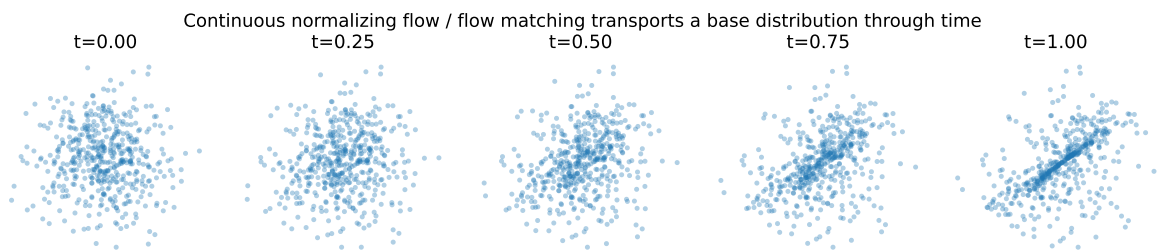
داده می‌شود. محاسبه trace در بعد بالا با Hutchinson تقریب زده می‌شود. آموزش CNF مبتنی بر likelihood ممکن است حل ODE در حلقه آموزش بخواهد. Matching Flow میدان سرعت یک مسیر احتمال انتخابی را با regression می‌آموزد، بدون حل ODE در هر گام آموزش. در مسیر خطی ساده

$$x_t = (1-t)x_0 + tx_1, \quad u_t = x_1 - x_0, \quad (۶۳.۱۲)$$

و هدف

$$\mathbb{E} \|v_\theta(x_t, t) - u_t\|^2 \quad (۶۴.۱۲)$$

است. مسیرهای انتقال بهینه می‌توانند trajectory مستقیم تر و sampling آسان تر بسازند. این بخش پلی مستقیم به فصل آینده diffusion و مدل‌های transport است.



شکل ۱۹.۱۲: انتقال پیوسته توزیع. مسیر احتمال و میدان سرعت دوشیء جدا هستند و انتخاب coupling روی دشواری regression اثر دارد.

کارگاه کد و آزمایش: Matching Flow دوبعدی

فایل `code/flow_matching_2d.py` یک میدان سرعت را روی interpolation میان Gaus-sian و هشت مد آموزش می‌دهد. برای ODE sampling، Euler و RK4 حل و اثر تعداد گام را بر coverage اندازه‌گیری کنید.

۱۷.۱۲ مدل‌های انرژی محور

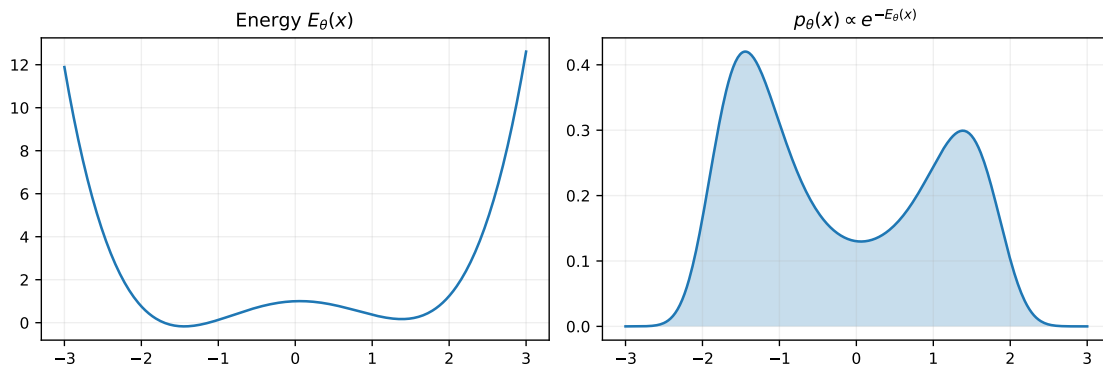
مدل انرژی

$$p_\theta(x) = \frac{\exp[-E_\theta(x)]}{Z_\theta}, \quad Z_\theta = \int \exp[-E_\theta(x)] dx \quad (۶۵.۱۲)$$

چگالی را تا ثابت نرمال‌سازی تعریف می‌کند. gradient درست‌نمایی

$$\nabla_\theta \log p_\theta(x) = -\nabla_\theta E_\theta(x) + \mathbb{E}_{x' \sim p_\theta} \nabla_\theta E_\theta(x') \quad (۶۶.۱۲)$$

دو فاز دارد: کاهش انرژی داده و افزایش انرژی نمونه مدل. فاز دوم به sampling از مدل نیاز دارد و دشوار است.



شکل ۲۰.۱۲: مدل انرژی محور: چگالی بالا در دره‌های انرژی است، اما ثابت نرمال‌سازی و نمونه‌گیری از کل منظره دشوارند.

۱۰.۱۷.۱۲ dynamics Langevin

نمونه‌گیری تقریبی با

$$x_{k+1} = x_k - \frac{\eta}{2} \nabla_x E_\theta(x_k) + \sqrt{\eta} \epsilon_k \quad (۶۷.۱۲)$$

انجام می‌شود. step، size تعداد گام، initialization و mixing روی bias نمونه اثر دارند. persistent chains هزینه را کم می‌کنند ولی خطاهای خود را دارند.

۲۰.۱۷.۱۲ matching Score

مدل score

$$s_\theta(x) = \nabla_x \log p_\theta(x) = -\nabla_x E_\theta(x) \quad (۶۸.۱۲)$$

است و ثابت نرمال‌سازی حذف می‌شود. matching score denoising روی داده نویزی پایه نظری مدل‌های score/diffusion است که در فصل بعد به‌طور کامل استخراج خواهد شد.

۱۸.۱۲ مدل‌های هیبرید و چرا مرز خانواده‌ها محو شده است

۱۰.۱۸.۱۲ VAE-GAN

encoder و VAE decoder با critic ادراکی ترکیب می‌شوند. زیان خصمانه جزئیات را تقویت می‌کند، اما ELBO دیگر تنها معیار مدل نیست و calibration چگالی مبهم‌تر می‌شود.

۲۰۱۸.۱۲ WAE و Autoencoder Adversarial

posterior تجمعی با discriminator یا IPM به prior نزدیک می‌شود. این مدل‌ها reconstruction و matching distribution را جدا می‌کنند.

۳۰۱۸.۱۲ dffusion latent و Transformer + VQGAN

autoencoder/VQ مدل فضای نهفته فشرده را می‌سازد و prior قدرتمند روی آن آموزش می‌بیند. در latent dffusion نیز AutoencoderKL تعیین می‌کند چه اطلاعاتی قبل از فرایند انتشار حذف شود؛ بنابراین VAE بخش حاشیه‌ای نیست، جزء معماری مولد مدرن است.

۴۰۱۸.۱۲ VAE در posterior Flow

posterior پایه با دنباله تبدیل‌های وارون‌پذیر غنی می‌شود:

$$\log q_K(z_K | x) = \log q_*(z_* | x) - \sum_{k=1}^K \log \left| \det \frac{\partial f_k}{\partial z_{k-1}} \right|. \quad (۶۹۰۱۲)$$

این کار شکاف واریاسیونی را کاهش می‌دهد، اما هزینه و خطر inference overfitting را افزایش می‌دهد.

۱۹۰۱۲ شرطی سازی، کنترل و ترجمه تصویر

شرط c می‌تواند label، متن، ماسک، edge، depth pose یا تصویر مرجع باشد. سه محل تزریق مهم‌اند:

۰۱. prior: $p(z | c)$

۰۲. شبکه مولد: normalization، cross-attention، concatenation یا modulation

۰۳. تابع هدف: contrastive classifier، perceptual alignment یا cycle loss.

۱۰۱۹۰۱۲ normalization Conditional

در BatchNorm conditional یا AdaIN، پارامترهای scale و shift تابع شرط‌اند:

$$y = \gamma(c) \odot \frac{x - \mu}{\sigma} + \beta(c). \quad (۷۰۰۱۲)$$

SPADE برای synthesis از modulation segmentation مکانی را حفظ می‌کند تا normalization اطلاعات layout را حذف نکند.

۲۰۱۹۰۱۲ تنوع شرطی

اگر $G(x, z)$ برای همه z خروجی مشابه دهد، مدل شرط را می‌آموزد ولی عدم قطعیت چندوجهی را نه. LPIPS میان خروجی‌های یک ورودی، regression latent و information mutual می‌توانند برای تشویق تنوع استفاده شوند؛ ولی باید fidelity شرطی نیز حفظ شود.

۲۰۰۱۲ فضای نهفته، درونیابی و جداسازی عوامل

۱۰۲۰۰۱۲ Linear و interpolation spherical

در Gaussian prior با بعد بالا، مسیر خطی بین دو نمونه ممکن است از ناحیه با norm معمول خارج شود. slerp مسیر کروی می‌سازد:

$$\text{slerp}(a, b; t) = \frac{\sin((1-t)\omega)}{\sin \omega} a + \frac{\sin(t\omega)}{\sin \omega} b, \quad (۷۱۰۱۲)$$

که ω زاویه دو بردار نرمال شده است. انتخاب مسیر باید با هندسه prior سازگار باشد.

کارگاه کد و آزمایش: درونیابی نهفته

فایل `norm code/latent_interpolation.py` مسیر linear و spherical را مقایسه می‌کند. آن را به decoder آموزش‌دیده وصل و تغییر، pose identity و texture را در طول مسیر جداگانه اندازه‌گیری کنید.

۲۰۲۰۰۱۲ Disentanglement بدون نظارت

جداسازی عوامل فقط با factorized prior تضمین نمی‌شود و بدون فرض استقرایی یا نظارت، شناسایی عمومی محدود است. معیارهایی مانند DCI MIG و SAP به labels عوامل وابسته‌اند و ممکن است با کیفیت بازسازی تعارض داشته باشند. بنابراین ادعای «هر بعد یک مفهوم» باید با مداخله و آزمون علی پشتیبانی شود.

۲۱۰۱۲ ارزیابی مدل مولد: یک عدد کافی نیست

۱۰۲۱۰۱۲ ELBO Likelihood و dimension per bits

برای مدل چگالی صریح،

$$\text{bits/dim} = -\frac{\log_2 p(x)}{HWC} \quad (۷۲۰۱۲)$$

گزارش می‌شود. برای VAE معمولاً bound یا estimate importance است. likelihood خوب می‌تواند به جزئیات کم سطح حساس باشد و OOD معنایی را تشخیص ندهد؛ بنابراین «چگالی بالا» مترادف «طبیعی و مرتبط» نیست.

FID ۲.۲۱.۱۲

اگر ویژگی‌های واقعی و مصنوعی Gaussian با میانگین‌های μ_r, μ_g و کوواریانس‌های C_r, C_g فرض شوند:

$$\text{FID} = \|\mu_r - \mu_g\|_2^2 + \text{trace} \left(C_r + C_g - 2(C_r^{1/2} C_g C_r^{1/2})^{1/2} \right). \quad (۷۳.۱۲)$$

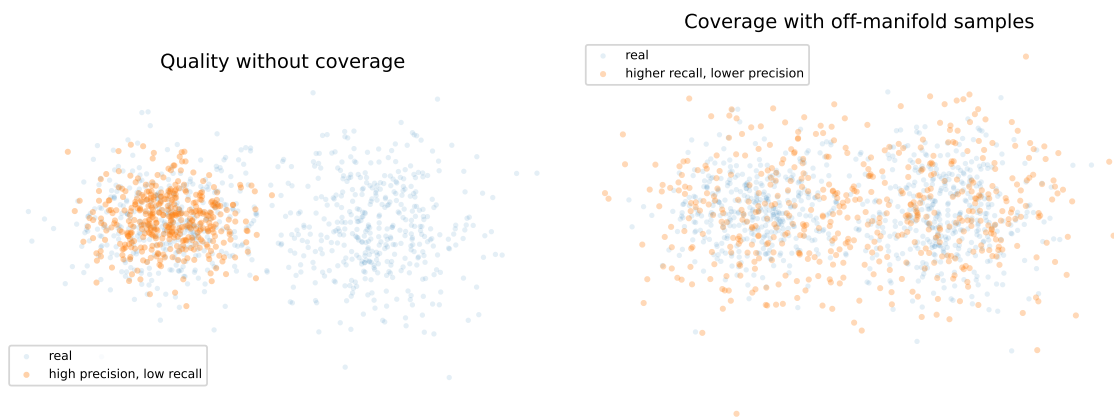
FID به تعداد نمونه، resize، preprocessing استخراج گر، خطای عددی sqrtm و reference dataset حساس و برآوردگر آن biased است. نسخه‌ها فقط با pipeline یکسان قابل مقایسه‌اند.

MMD و KID ۳.۲۱.۱۲

KID از MMD مربعی با kernel چندجمله‌ای روی ویژگی‌ها استفاده و برآوردگر بدون اریب ساده‌ای دارد. در نمونه کم می‌تواند مفیدتر باشد، ولی همچنان به embedding انتخابی وابسته است.

Recall و Precision ۴.۲۱.۱۲ توزیعی

precision تقریباً سهم نمونه‌های تولیدی نزدیک منیفولد واقعی و recall سهم داده واقعی پوشش داده شده توسط مولد را می‌سنجد. truncation معمولاً precision را بالا و recall را پایین می‌برد.



شکل ۲۱.۱۲: کیفیت و پوشش دو محور متفاوت‌اند. FID می‌تواند این دو را در یک عدد مخلوط کند.

۵.۲۱.۱۲ حفظ کردن داده و audit nearest-neighbor

برای هر نمونه مصنوعی نزدیک‌ترین نمونه آموزش در چند فضای perceptual pixel و feature foundation را بیابید. شباهت زیاد به تنهایی اثبات کپی نیست، اما باید با عضویت، detection duplicate و تغییرات

augmentation بررسی شود. در داده پزشکی یا خصوصی، ارزیابی privacy و memorization ضروری است.

۶.۲۱.۱۲ ارزیابی شرطی

برای text-to-image یا label-to-image باید هم fidelity شرط و هم realism/coverage سنجیده شود. CLIPScore می‌تواند alignment را بسنجد ولی نسبت به encoder bias و shortcut حساس است. برای داده علی، metric دامنه‌ای و task downstream غالباً معتبرتر از Inception عمومی است.

کارگاه کد و آزمایش: FID و KID روی ویژگی مصنوعی

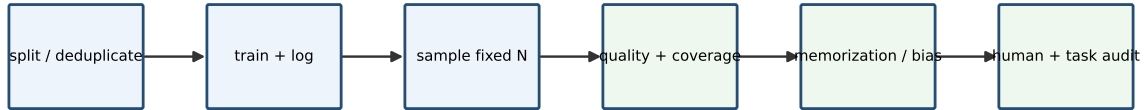
```

1 from __future__ import annotations
2 import numpy as np
3
4 def sqrtm_psd(a: np.ndarray) -> np.ndarray:
5     w,v=np.linalg.eigh((a+a.T)/2); return (v*np.sqrt(np.clip(w,0,None)
6     ))@v.T
7
8 def fid(x: np.ndarray,y: np.ndarray) -> float:
9     mx,my=x.mean(0),y.mean(0); cx=np.cov(x,rowvar=False); cy=np.cov(y,
10    rowvar=False)
11     cross=sqrtm_psd(sqrtm_psd(cx)@cy@sqrtm_psd(cx))
12     return float(np.sum((mx-my)**2)+np.trace(cx+cy-2*cross))
13
14 def kid_unbiased(x: np.ndarray,y: np.ndarray) -> float:
15     d=x.shape[1]
16     kxx=(x@x.T/d+1)**3; kyy=(y@y.T/d+1)**3; kxy=(x@y.T/d+1)**3
17     m,n=len(x),len(y)
18     return float((kxx.sum()-np.trace(kxx))/(m*(m-1))+(kyy.sum()-np.
19     trace(kyy))/(n*(n-1))-2*kxy.mean())
20
21 if __name__=='__main__':
22     rng=np.random.default_rng(0); real=rng.normal(size=(500,64)); good
23     =rng.normal(.05,1.03,size=(500,64)); bad=rng.normal(.5,1.4,size
24     =(500,64))
25     print({"fid_good":fid(real,good),"fid_bad":fid(real,bad),"kid_good
26     ":kid_unbiased(real,good),"kid_bad":kid_unbiased(real,bad)})

```

آزمایش کنید با $N = 100, 500, 5000$ رتبه دو مدل چقدر پایدار است. سپس embedding را با تبدیل خطی و غیرخطی تغییر دهید و نشان دهید metric به فضای ویژگی وابسته است.

No single number is a sufficient evaluation of a generative model



Report likelihood/ELBO when available, FID/KID, precision-recall, nearest-neighbor audits, conditional fidelity, cost, and failure cases.

شکل ۲۲.۱۲: پروتکل ارزیابی چندمحوری. جداسازی داده، تعداد نمونه ثابت و ممیزی شکست قبل از انتخاب بهترین checkpoint ضروری است.

۲۲.۱۲ پروتکل آزمایش بازتولیدپذیر

۱. داده: منبع، مجوز، deduplication، رزولوشن، crop و گروه‌های جمعیتی ثبت شود؛
۲. **split**: train/validation/test و هرگونه overlap با pretraining مشخص شود؛
۳. **compute**: GPU/CPU، precision، batch، accumulation، زمان و انرژی تقریبی گزارش شود؛
۴. آموزش: seed، optimizer، نرخ‌های دو بازیکن، EMA، augmentation و rule checkpoint ثبت شود؛
۵. **sampling**: تعداد، seed، truncation، top-k/p temperature و guidance ثابت باشد؛
۶. **metric**: نسخه، feature، preprocessing extractor و statistics reference ذخیره شود؛
۷. عدم قطعیت: چند run یا confidence bootstrap، interval؛
۸. تصویرها: نمونه تصادفی و failure، gallery، نه فقط cherry-picked؛
۹. ممیزی: nearest، neighbor، memorization، artifact bias و conditional، failure؛
۱۰. کد: commit، محیط، checksum و log خام نگهداری شود.

۲۳.۱۲ کتابخانه‌ها و ابزارهای حرفه‌ای

Library stack: mathematics first, framework second, pretrained systems last

Research models	StyleGAN3	taming-transformers	VAR/LlamaGen
Generative toolboxes	MMagic	Diffusers VAE/VQ	MONAI Generative
Metrics and tracking	torch-fidelity	Clean-FID	W&B / MLflow
Core	PyTorch	torch.distributions	torchvision

شکل ۲۳.۱۲: پشته کتابخانه‌ای. استفاده از toolbox نباید قراردادهای normalization، likelihood، sampling و metric را پنهان کند.

۱.۲۳.۱۲ PyTorch و torch.distributions

برای reparameterized distribution، distribution transformed sampling و constraint مناسب است. تفاوت sample() و rsample() باید فهمیده شود؛ دومی gradient pathwise دارد اگر توزیع پشتیبانی کند.

۲.۲۳.۱۲ torchvision tutorial DCGAN

برای baseline آموزشی خوب است و pipeline داده، initialization و حلقه دو optimizer را شفاف نشان می‌دهد. برای پژوهش، باید metric، چند augmentation seed و checkpointing اضافه شود.

۳.۲۳.۱۲ MMagic

ابزار OpenMMLab برای generation و restoration با registry، runner config و zoo model است. مزیت آن مقایسه هاست؛ pipeline خطر آن استفاده از config بدون فهم preprocessing و loss است.

۴۰.۲۳.۱۲ مخزن رسمی StyleGAN ۳

برای آموزش، تولید، projection و های checkpoint رسمی استفاده می‌شود. قرارداد dataset، ZIP، رزولوشن، label، augmentation metadata و گزینه‌های alias-free باید دقیق رعایت شود.

۵۰.۲۳.۱۲ VQModel و AutoencoderKL Dffusers

حتی پیش از فصل، dffusion این کلاس‌ها برای encoder/decoder نهفته مدرن مهم‌اند. factor، scaling، latent، posterior sample channels، mode و preprocessing تصویر باید ثبت شوند.

۶۰.۲۳.۱۲ ابزار معیار

torch-fidelity و Clean-FID اجرای معیار را استانداردتر می‌کنند، اما reference نسخه وزن و resize همچنان باید ذکر شوند. metric را در environment قفل کنید و آمار واقعی را دوباره استفاده کنید.

۲۴.۱۲ انتخاب خانواده مناسب

خانواده	چگالی	نمونه‌گیری	استنباط $z x$	شکست شاخص
VAE	کران واریاسیونی	موازی و سریع	استنباط تعمیم‌یافته	تاری / فروپاشی پسین
VQ-VAE	با وابسته به پیشین	ترتیبی	رمزگذار کوانتیزه	گلوگاه توکن ساز و کدبوک
GAN	ضمنی	یک گذر روبه جلو	نیازمند وارون‌سازی	ناپایداری و فروپاشی مد
خودرگرسیو	دقیق	ترتیبی	معمولاً مستقیم	کندی نمونه‌گیری / ترتیب مصنوعی
جریان نرمال‌ساز	دقیق	دقیق و وارون	دقیق	قید وارون‌پذیری و حافظه
انرژی‌محور	تا ثابت Z	MCMC	بهینه‌سازی نمونه‌گیری	یا اختلاط ضعیف و ثابت نرمال‌سازی
تطبیق جریان	مسیر پیوسته	حل ODE	وارون‌سازی ODE	با خطای حل گر و انتخاب مسیر

اصل راهنما

خانواده را بر اساس نیاز انتخاب کنید، نه شهرت: اگر likelihood anomaly مهم است، density و calibration را بسنجید؛ اگر generation latency مهم است، GAN یا decoder موازی؛ اگر

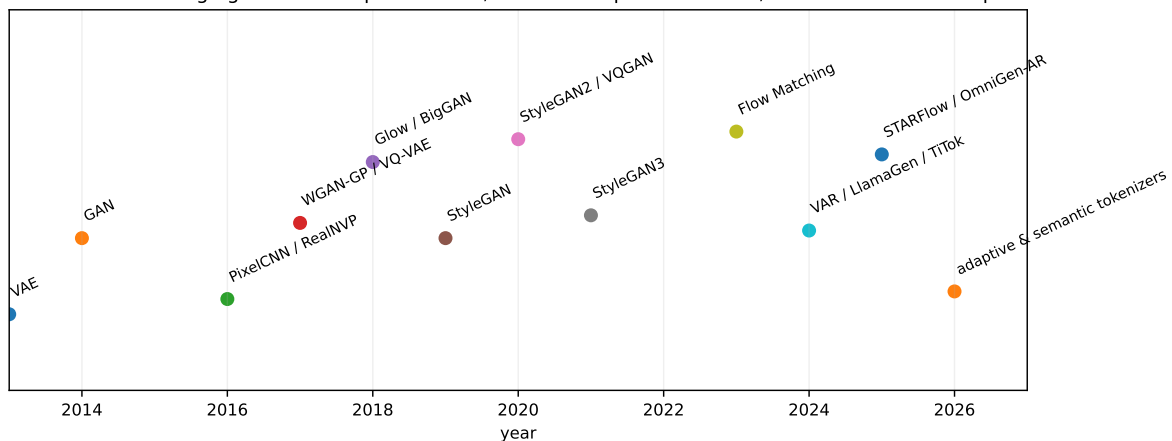
reconstruction و representation مهم است، به VAE/VQ اگر infrastructure sequence و scaling مهم است، token، AR، و اگر transport پیوسته و guidance مهم است، flow/diffusion.

۲۵.۱۲ افق ۲۰۲۴ تا ۲۰۲۶

پژوهش مولد تصویر از دوگانه قدیمی GAN در برابر VAE عبور کرده است. چند روند اصلی:

- های tokenizer بسیار فشرده یک‌بعدی مانند TiTok و نسخه‌های adaptive/semantic
- بازگشت مدل‌های خودرگرسیو با LlamaGen، autoregression set/scale VAR، و زیرساخت LLM
- جریان‌های نهفته مقیاس‌پذیر و flow matching
- مدل‌های هیبرید AR-diffusion و tokenizer-foundation feature
- توجه بیشتر به کیفیت dense، dimension codebook latent، و rate-distortion
- ارزیابی با featureهای extractor دامنه‌ای و ممیزی حفظ کردن داده
- watermark provenance و تشخیص رسانه مصنوعی به عنوان جزء سامانه، نه افزونه بعدی.

The field is converging toward compact latents, scalable sequence models, and continuous transport



شکل ۲۴.۱۲: خط زمانی مفهومی. مرزهای خانواده‌ها با های tokenizer فشرده، های prior ترتیبی و انتقال پیوسته در حال همگرایی‌اند.

پل به دهه آینده

فصل سیزدهم مدل‌های انتشار، score، SDE/ODE، latent denoising، match-flow diffusion، ing و sampling سریع را از ابتدا و با ریاضیات کامل توسعه می‌دهد. در فصل حاضر فقط مفاهیمی آمده‌اند که برای فهم جایگاه آن خانواده در نقشه مولدها لازم است.

۲۶.۱۲ الگوریتم‌های مرجع فصل

الگوریتم ۳۰.۱۲ - VQ-VAE با EMA codebook

۱. $z_e = E(x)$ ؛ assignment نزدیک‌ترین؛ code
۲. decoder روی quantized straight-through؛ latent
۳. reconstruction و commitment؛ loss
۴. برای هر code، شمارش و مجموع encoderها؛ vector
۵. EMA شمارش و مجموع؛ $e_k \leftarrow m_k/N_k$
۶. reset کدهای مرده با نمونه؛ encoder
۷. ثابت، perplexity درصد کد فعال، rank effective و rFID.

الگوریتم ۴۰.۱۲ - GAN پایدار با ۱R و EMA

۱. update تمایزگر با logistic/hinge؛ loss
۲. هر r گام، ۱R روی داده واقعی با ضریب تصحیح شده؛
۳. update مولد با non-saturating یا hinge
۴. EMA وزن مولد؛
۵. augmentation تطبیقی در داده محدود؛
۶. هر بازه ثابت، نمونه با seed ثابت و metric با N ثابت؛
۷. توقف بر اساس metric validation و coverage، نه loss training تنها.

الگوریتم ۵۰.۱۲ - نمونه‌گیری autoregressive با temperature و top-k

- برای $i = 1$ تا N :
۱. $\ell_i = f(c_{<i}, condition)$ logits
۲. $\ell_i \leftarrow \ell_i/T$
۳. همه logits خارج top-k را $-\infty$ کنید؛
۴. $c_i \sim \text{Categorical}(\text{softmax}(\ell_i))$
۵. token را به cache اضافه کنید؛

۶. در پایان، decoder ها token را به تصویر تبدیل کند.
temperature کم precision را بالا و diversity را کم می‌کند؛ اثر باید گزارش شود.

الگوریتم ۶.۱۲ - آموزش RealNVP با likelihood maximum

۱. dequantize و transform logit با Jacobian، correction

۲. $z = f_{\theta}(x)$ و logdet همه ها؛ coupling

۳. $\log p(x) = \log p_Z(z) + \sum \log \det$

۴. منفی میانگین log-likelihood را کمینه کنید؛

۵. آزمون وارون و logdet عددی در هر تغییر معماری؛

۶. bits/dim و نمونه prior را هر دو گزارش کنید.

۲۷.۱۲ آزمایش‌های پیشنهادی فصل

۱. **rate-distortion: VAE** $\beta \in \{0.1, 0.5, 1, 2, 4\}$ ، recon، KL active units و FID نمونه prior.

۲. **collapse: Posterior** decoder ضعیف/قوی، KL warm-up و bits. free

۳. **codebook: VQ** اندازه codebook، بعد EMA code، و rFID reset در برابر perplexity.

۴. **objective: GAN**، minimax، hinge non-saturating و WGAN-GP روی مخلوط مدها.

۵. **Lipschitz:** GP clipping و spectral، norm norm در فضای ورودی.

۶. **GAN: Data-limited** بدون DfFiAugment augmentation و ADA.

۷. **Truncation:** منحنی precision-recall و FID در چند ψ .

۸. **order: Autoregressive**، serpentine raster و coarse-to-fine.

۹. **frontier: Tokenizer** count token در برابر reconstruction و زمان generation.

۱۰. **Flow:** تعداد coupling و نوع، likelihood permutation در برابر sample quality.

۱۱. **stability: Metric** FID/KID با تعداد نمونه و extractor مختلف.

۱۲. **audit: Memorization** نزدیک‌ترین همسایه train/test برای چند checkpoint.

۲۸.۱۲ پروژه‌های پایان فصل

۱. آزمایشگاه VAE تشخیصی: dashboard نرخ، اعوجاج، units active و collapse. posterior
۲. tokenizer VQ برای تصاویر صنعتی: مقایسه، VQ residual codebook و tokenizer پیوسته.
۳. GAN داده کم: ADA^۲StyleGAN- یا baseline کوچک با ممیزی. overfit.
۴. ترجمه تصویر علمی: pix/CycleGAN^۲pix با قید حفظ ساختار و آزمون. hallucination.
۵. مولد توکنی کوچک: Transformer + VQ-VAE روی تصاویر ۳۲×۳۲ .
۶. RealNVP/Glow آموزشی: چگالی، interpolation sampling و OOD. paradox.
۷. سامانه معیار: KID، FID، neighbor nearest precision/recall و گزارش HTML.
۸. مطالعه خانواده‌ها: compute-matched، GAN VAE و AR با بودجه پارامتر و زمان یکسان.

۲۹.۱۲ تمرین‌های نظری

۱. هویت دقیق، ELBO log-likelihood و gap variational را اثبات کنید.
۲. KL دو Gaussian قطری را از تعریف استخراج کنید.
۳. نشان دهید MSE معادل منفی Gaussian log-likelihood با واریانس ثابت است.
۴. برای decoder Gaussian با واریانس آموختنی، تابع زیان کامل را بنویسید.
۵. gradient بازپارامتردهی برای $h(z) = z^2$ را تحلیل و مونت کارلو مقایسه کنید.
۶. posterior aggregate را تعریف و تفاوت WAE با VAE را توضیح دهید.
۷. تمایزگر بهینه GAN را استخراج و مقدار $V(D^*, G)$ را به JS تبدیل کنید.
۸. توضیح دهید چرا loss non-saturating همان divergence ساده را دقیقاً کمینه نمی‌کند.
۹. دوگان Wasserstein-۱ را بیان و نقش Lipschitz را توضیح دهید.
۱۰. یک مثال بسازید که clipping weight ظرفیت critic را محدود کند.
۱۱. affine coupling logdet را استخراج کنید.
۱۲. رابطه میان MAF و IAF را از وارون کردن ترتیب توضیح دهید.
۱۳. نشان دهید convolution masked نوع A leakage ندارد.
۱۴. برای تصویر RGB یک فاکتورگیری علی میان پیکسل و کانال طراحی کنید.
۱۵. bias برآوردگر FID را با شبیه‌سازی بررسی کنید.

- ۱۶. نشان دهید truncation می‌تواند recall را کاهش دهد.
- ۱۷. یک counterexample برای تفسیر likelihood به عنوان score OOD بسازید.
- ۱۸. تفاوت perplexity codebook و rank effective را توضیح دهید.
- ۱۹. چرا consistency cycle شناسایی معنایی را تضمین نمی‌کند؟
- ۲۰. matching flow و CNF likelihood maximum را از نظر نیاز به حل ODE مقایسه کنید.

۳۰۰۱۲ تمرین‌های کدنویسی

- ۱. به encoder/decoder convolutional VAE، اضافه کنید.
- ۲. per-dimension KL و units active را log کنید.
- ۳. bits free و annealing cyclical را پیاده کنید.
- ۴. VQ با EMA و reset که مرده بسازید.
- ۵. penalty gradient را با difference finite کنترل کنید.
- ۶. normalization spectral را با iteration power از صفر پیاده کنید.
- ۷. روی occupancy GAN، هشت مد و entropy آن را رسم کنید.
- ۸. lazy اجرا و ضرب درست را آزمون کنید.
- ۹. convolution masked را برای ordering channel RGB توسعه دهید.
- ۱۰. KV-cache ساده برای prior token Transformer بسازید.
- ۱۱. RealNVP را روی two-moons آموزش دهید.
- ۱۲. analytic logdet و Jacobian autograd را مقایسه کنید.
- ۱۳. FID را با interval bootstrap گزارش کنید.
- ۱۴. precision/recall توزیعی را روی feature مصنوعی پیاده کنید.
- ۱۵. train/test montage nearest-neighbor تولید کنید.

تقسیم پیشنهادی ویدئوهای ۱۵ دقیقه‌ای

پیشنهاد فصل برای ۱۰۴ ویدئوی ۱۵ دقیقه‌ای:

- ۱. ویدئوهای ۱-۸: تعریف مدل مولد، likelihood pushforward و واگرایی‌ها؛
- ۲. ۹-۲۴: خودرمزگذار، ELBO، بازپارامتردهی، likelihood مشاهده و کد VAE؛
- ۳. ۲۵-۳۴: β -VAE، collapse، posterior، WAE IWAE و VAE سلسله‌مراتبی؛

۰۴. ۳۵-۴۴: VQGAN codebook، VQ-VAE، و آزمایش، tokenizer،
۰۵. ۴۵-۶۲: GAN، تمایزگر بهینه، زیان‌ها، دینامیک، WGAN collapse، و regularization،
۰۶. ۶۳-۷۴: DCGAN، pix، ۲pix conditional، BigGAN CycleGAN، و StyleGAN،
۰۷. ۷۵-۸۴: PixelCNN، likelihood پیکسل، token، prior، LlamaGen MaskGIT، و VAR،
۰۸. ۸۵-۹۳: MAF/IAF، Glow، RealNVP، CNF dequantization، و flow، matching،
۰۹. ۹۴-۹۸: Langevin EBM، و score،
۰۱۰. ۹۹-۱۰۲: معیارها، میزی، کتابخانه‌ها و بازتولید،
۰۱۱. ۱۰۳-۱۰۴: ارائه پروژه و پل به dffusion.

۳۱۰۱۲ پرامپت جامع برای Codex

قرارداد میان ریاضیات و کد

کد باید از متن علمی تبعیت کند: هر loss با رابطه فصل، هر metric با نسخه و preprocessing، هر نمودار با CSV خام و هر checkpoint با config و seed همراه باشد. خروجی تصویری بدون log عددی نتیجه علمی محسوب نمی‌شود.

- 1 Act **as** a senior research engineer **in** deep generative image modeling.
- 2 Build a reproducible PyTorch laboratory comparing six families under a
- 3 matched compute budget: VAE, VQ-VAE, non-saturating GAN, WGAN-GP,
- 4 PixelCNN/token autoregressive model, **and** RealNVP.
- 5
- 6 Requirements:
- 7 1. Use a single configuration system **and** deterministic seeds.
- 8 2. Provide synthetic 2-D sanity tests before image experiments.
- 9 3. Separate tokenizer reconstruction quality **from** token-prior quality.
- 10 4. Log reconstruction, KL/rate, active units, code usage/perplexity,
- 11 critic gradients, mode occupancy, likelihood/bits-dim, FID, KID,
- 12 precision-recall, latency, memory, **and** nearest-neighbor audits.
- 13 5. Implement unit tests **for** VAE reparameterization shapes, masked-conv
- 14 causality, flow invertibility/log-det, gradient penalty, **and** metric
- 15 invariance to batch partitioning.
- 16 6. Never invent results. Every table must be generated **from** CSV/JSON logs.
- 17 7. Save random **and** failure samples, **not** only curated samples.
- 18 8. Export an experiment card **with** dataset license, preprocessing,
- 19 model commit, hardware, runtime, seeds, sampling parameters, **and** caveats.

- 20 9. Include README commands **for** CPU smoke tests **and** GPU full runs.
- 21 10. Produce a LaTeX-ready table **and** figure manifest.

جمع‌بندی فصل

- VAE یک مدل متغیر نهفته با استنباط amortized و کران واریاسیونی است؛ reconstruction و KL معنای احتمالاتی دارند.
- VQ-VAE تصویر را به الفبای گسسته تبدیل می‌کند؛ کیفیت و استفاده codebook باید مستقل از prior سنجیده شود.
- GAN یک بازی توزیعی با چگالی ضمنی است؛ کیفیت نمونه بالا می‌تواند با پوشش ناقص همراه باشد.
- WGAN و های Lipschitz regularization هندسه gradient را بهبود می‌دهند ولی جای عیب‌یابی را نمی‌گیرند.
- مدل خودرگرسیو likelihood دقیق دارد، اما ترتیب و sampling متوالی هزینه‌زاست؛ tok-enization و next-scale این هزینه را تغییر می‌دهند.
- جریان نرمال‌ساز likelihood و inversion دقیق را با قید وارون‌پذیری و determinant قابل محاسبه مبادله می‌کند.
- مدل انرژی‌محور انعطاف‌پذیر است، ولی sampling و ثابت نرمال‌سازی دشوارند.
- هیچ معیار واحدی کافی نیست؛ fidelity، memorization، coverage، quality، likelihood، شرطی و compute باید با هم گزارش شوند.

منابع منتخب و مسیر مطالعه

Bibliography

- [1] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” ICLR, 2014.
- [2] D. P. Kingma and M. Welling, “An Introduction to Variational Autoencoders,” Foundations and Trends in Machine Learning, 2019.
- [3] Y. Burda, R. Grosse, and R. Salakhutdinov, “Importance Weighted Autoencoders,” ICLR, 2016.
- [4] I. Higgins et al., “beta-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework,” ICLR, 2017.
- [5] H. Kim and A. Mnih, “Disentangling by Factorising,” ICML, 2018.
- [6] S. Zhao, J. Song, and S. Ermon, “InfoVAE: Information Maximizing Variational Autoencoders,” AAAI, 2019.
- [7] I. Tolstikhin et al., “Wasserstein Auto-Encoders,” ICLR, 2018.
- [8] A. Vahdat and J. Kautz, “NVAE: A Deep Hierarchical Variational Autoencoder,” NeurIPS, 2020.
- [9] R. Child, “Very Deep VAEs Generalize Autoregressive Models and Can Outperform Them on Images,” ICLR, 2021.
- [10] A. van den Oord, O. Vinyals, and K. Kavukcuoglu, “Neural Discrete Representation Learning,” NeurIPS, 2017.
- [11] P. Esser, R. Rombach, and B. Ommer, “Taming Transformers for High-Resolution Image Synthesis,” CVPR, 2021.
- [12] I. Goodfellow et al., “Generative Adversarial Nets,” NeurIPS, 2014.
- [13] A. Radford, L. Metz, and S. Chintala, “Unsupervised Representation Learning with Deep Convolutional GANs,” ICLR, 2016.

- [14] S. Nowozin, B. Cseke, and R. Tomioka, “f-GAN: Training Generative Neural Samplers using Variational Divergence Minimization,” *NeurIPS*, 2016.
- [15] M. Arjovsky, S. Chintala, and L. Bottou, “Wasserstein GAN,” *ICML*, 2017.
- [16] I. Gulrajani et al., “Improved Training of Wasserstein GANs,” *NeurIPS*, 2017.
- [17] T. Miyato et al., “Spectral Normalization for Generative Adversarial Networks,” *ICLR*, 2018.
- [18] M. Heusel et al., “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium,” *NeurIPS*, 2017.
- [19] A. Brock, J. Donahue, and K. Simonyan, “Large Scale GAN Training for High Fidelity Natural Image Synthesis,” *ICLR*, 2019.
- [20] T. Karras, S. Laine, and T. Aila, “A Style-Based Generator Architecture for Generative Adversarial Networks,” *CVPR*, 2019.
- [21] T. Karras et al., “Analyzing and Improving the Image Quality of StyleGAN,” *CVPR*, 2020.
- [22] T. Karras et al., “Training Generative Adversarial Networks with Limited Data,” *NeurIPS*, 2020.
- [23] T. Karras et al., “Alias-Free Generative Adversarial Networks,” *NeurIPS*, 2021.
- [24] P. Isola et al., “Image-to-Image Translation with Conditional Adversarial Networks,” *CVPR*, 2017.
- [25] J.-Y. Zhu et al., “Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks,” *ICCV*, 2017.
- [26] A. van den Oord et al., “Conditional Image Generation with PixelCNN Decoders,” *NeurIPS*, 2016.
- [27] T. Salimans et al., “PixelCNN++: Improving the PixelCNN with Discretized Logistic Mixture Likelihood and Other Modifications,” *ICLR*, 2017.
- [28] L. Dinh, J. Sohl-Dickstein, and S. Bengio, “Density Estimation using Real NVP,” *ICLR*, 2017.

- [29] G. Papamakarios, T. Pavlakou, and I. Murray, “Masked Autoregressive Flow for Density Estimation,” *NeurIPS*, 2017.
- [30] D. P. Kingma et al., “Improved Variational Inference with Inverse Autoregressive Flow,” *NeurIPS*, 2016.
- [31] D. P. Kingma and P. Dhariwal, “Glow: Generative Flow with Invertible 1x1 Convolutions,” *NeurIPS*, 2018.
- [32] W. Grathwohl et al., “FFJORD: Free-form Continuous Dynamics for Scalable Reversible Generative Models,” *ICLR*, 2019.
- [33] Y. Lipman et al., “Flow Matching for Generative Modeling,” *ICLR*, 2023.
- [34] M. Heusel et al., “GANs Trained by a Two Time-Scale Update Rule...,” *NeurIPS*, 2017.
- [35] M. Binkowski et al., “Demystifying MMD GANs,” *ICLR*, 2018.
- [36] M. Sajjadi et al., “Assessing Generative Models via Precision and Recall,” *NeurIPS*, 2018.
- [37] T. Kynkaanniemi et al., “Improved Precision and Recall Metric for Assessing Generative Models,” *NeurIPS*, 2019.
- [38] K. Tian et al., “Visual Autoregressive Modeling: Scalable Image Generation via Next-Scale Prediction,” *NeurIPS*, 2024.
- [39] P. Sun et al., “Autoregressive Model Beats Diffusion: Llama for Scalable Image Generation,” 2024.
- [40] Q. Yu et al., “An Image is Worth 32 Tokens for Reconstruction and Generation,” *NeurIPS*, 2024.
- [41] “OmniGen-AR: AutoRegressive Any-to-Image Generation,” *NeurIPS*, 2025.
- [42] “Scaling Latent Normalizing Flows for High-Resolution Image Synthesis,” *NeurIPS*, 2025.
- [43] X. Zhao et al., “Continuous First, Discrete Later: VQ-VAEs Without Dimensional Collapse,” 2026.
- [44] PyTorch Contributors, “DCGAN Tutorial,” official documentation, accessed July 2026.

- [45] Hugging Face Contributors, “AutoencoderKL and VQModel Documentation,” official Diffusers documentation, accessed July 2026.
- [46] OpenMMLab Contributors, “MAGIC Documentation and Model Zoo,” accessed July 2026.
- [47] NVIDIA Research, “StyleGAN3 Official PyTorch Implementation,” accessed July 2026.

پیوست آ

راهنمای ساخت، اجرا و بازتولید

آ.۱ ساخت PDF

```
1 xelatex image_processing_next_decade_ch12_fa.tex
2 xelatex image_processing_next_decade_ch12_fa.tex
```

آ.۲ محیط پایتون

```
1 python -m venv .venv
2 source .venv/bin/activate          # Windows: .venv\Scripts\activate
3 python -m pip install --upgrade pip
4 python -m pip install -r requirements.txt
```

آ.۳ تولید شکل‌ها

```
1 python generate_figures.py
```

تمام نمودارهای عددی فصل آموزشی و بازتولیدپذیرند و به عنوان نتیجه تجربی مقاله معرفی نشده‌اند.

آ.۴ test Smoke کدها

```
1 python code/vae_minimal.py
2 python code/vq_vae_minimal.py
3 python code/gan_minimal.py
4 python code/wgan_gp_minimal.py
5 python code/realnvp_2d.py
6 python code/pixelcnn_mask.py
```

```
7 python code/fid_kid_toy.py
8 python code/latent_interpolation.py
9 python code/flow_matching_2d.py
```

خروجی اجرای کنترل‌شده در `outputs/smoke_test.txt` ذخیره شده است.

آ.۵ پیش‌پرواز علمی

۰۱. خروجی‌های نمونه با `seed` و `checkpoint` مشخص باشند؛
۰۲. بهترین نمونه‌ها جدا از نمونه تصادفی برچسب بخورند؛
۰۳. `metric` روی تعداد نمونه و `preprocessing` ثابت محاسبه شود؛
۰۴. `tokenizer` و `prior` جداگانه ارزیابی شوند؛
۰۵. `neighbor nearest train/test` هر دو بررسی شوند؛
۰۶. `seed` یا `uncertainty bootstrap` ارائه شود؛
۰۷. `collapse failure` و `subgroup` ضعیف پنهان نشوند؛
۰۸. `license` داده و وزن‌ها ثبت شود.